УНИВЕРСИТЕТ ПУЧОН В ГОРОДЕ ТАШКЕНТЕ
Кафедра e-Business
Программная инженерия · 3 курс · 5 семестр

ПРОЕКТНАЯ РАБОТА

Главы 1–3

«Предзаказ обеда в университетской столовой»

Промежуточный контроль · этапы 1–3 · темы 1–7

Команда 1
A — Шашкина Виктория: Product Owner, бизнес-аналитик
B — Алиев Салим: системный аналитик, UX-дизайнер
C — Фаткулбаянов Руслан (капитан): менеджер проекта, QA-инженер

Преподаватель: Ботиров С. Р.

Репозиторий: https://github.com/Zombik333/team1-predzakaz-obeda
Прототип (Figma): ⟦ссылка на Figma — добавит Салим⟧
Версия: 1.1 (сокращённая) · 03.11.2026

Ташкент — 2026

Содержание

Введение ... 3
Глава 1. Идея и организация проекта ... 4
1.1. Устав команды ... 4
1.2. Product One-Pager и матрица отбора идеи ... 4
1.3. Бизнес-модель и юнит-экономика ... 6
1.4. Устав проекта и модель жизненного цикла ... 8
Выводы по главе 1 ... 9
Глава 2. Требования ... 10
2.1. Видение и границы продукта (Vision & Scope) ... 10
2.2. Заинтересованные стороны ... 10
2.3. Исследование: интервью, опрос и замеры ... 10
2.4. Персоны и карта пути ... 11
2.5. Спецификация требований (SRS), версия 2.1 ... 11
2.6. Бэклог и приоритеты ... 13
2.7. Матрица трассируемости ... 14
Выводы по главе 2 ... 15
Глава 3. Проектирование и выбор платформы ... 16
3.1. Бизнес-процессы ... 16
3.2. UML-модели ... 20
3.3. Модель данных ... 24
3.4. Архитектура ... 25
3.5. Прототип интерфейса ... 28
3.6. Выбор платформы и стоимость владения ... 29
Выводы по главе 3 ... 31
Заключение ... 33
Список использованных источников ... 33
Приложение А. Протоколы интервью (модельные данные) ... 35
Приложение Б. Опрос (модельные данные) ... 36
Приложение В. Функциональные требования ... 36
Приложение Г. Описание ключевых таблиц базы данных ... 37
Приложение Д. Сценарий проверки прототипа ... 38
Приложение Е. Организация работы команды ... 38
Приложение Ж. Ссылки и исходные файлы ... 39
Приложение И. История изменений ... 39

Введение

Работа описывает сервис предзаказа обеда в университетской столовой. Студент заранее выбирает блюдо и перемену, платит через Payme, Click или Uzum и забирает обед по коду у отдельной точки выдачи. Столовая получает список заказов по переменам и учёт каждого оплаченного заказа. Цель продукта (BG-0): за 3 месяца после запуска студент получает обед в большую перемену без очереди, а столовая ведёт учёт каждого онлайн-заказа.

Работа использует два вида данных. Реальные данные — опрос трёх однокурсников 09.10.2026, оценки команды, проверка конкурентов 07.10.2026 и расписание университета. Модельные данные — интервью П1–П5, опрос В1–В10, замеры очереди и проверка прототипа: команда составила их сама по опыту участников и по данным этапа 1. Каждое модельное число в тексте имеет пометку «модельные данные», а план их замены реальными данными дан в заключении.

Глава 1. Идея и организация проекта

Глава 1 отвечает на два вопроса: какую проблему решает продукт и как команда организует работу над ним. Это первый шаг работы: без подтверждённой проблемы и без бизнес-модели у требований главы 2 нет основы. Глава содержит устав команды, Product One-Pager и матрицу отбора идеи, бизнес-модель с юнит-экономикой и устав проекта с выбором модели жизненного цикла. Раздел 1.3 команда добавила после темы 3. Остальные разделы — версия этапа 1 с исправлениями; каждое исправление записано в приложении И.

1.1. Устав команды

Устав команды распределяет роли из задания этапа 1. Шашкина Виктория (А) — Product Owner и бизнес-аналитик: она ведёт линию «проблема → цель → история» от One-Pager до бэклога и процессов. Алиев Салим (В) — системный аналитик и UX-дизайнер: он ведёт линию «пользователь → данные → интерфейс», то есть персоны, схему данных и прототип. Фаткулбаянов Руслан (С, капитан) — менеджер проекта и QA-инженер: он ведёт проверку требований, матрицу трассируемости и расчёт стоимости владения. Автор раздела и проверяющий — разные люди: требования пишет участник А, а проверяет участник С. Правила работы (канал связи, встречи, решения, срыв срока, определение готовности) даны в приложении Е.

1.2. Product One-Pager и матрица отбора идеи

Product One-Pager описывает продукт на одной странице: проблему, пользователей, ценность, функции, бизнес-модель, конкурентов, границы и метрику. Таблица 1.1 показывает версию к промежуточному контролю. По сравнению с этапом 1 изменена строка «Бизнес-модель»: команда выбрала вариант по результатам интервью П5 (раздел 2.3).

Таблица 1.1 — Product One-Pager

Поле	Содержание
Проблема	В большую перемену у стойки столовой собирается толпа. По оценке команды, ожидание доходит до 6–7 минут, а малая перемена длится 10 минут. Популярные блюда заканчиваются до большой перемены. Оплата идёт наличными или переводом на карту сотрудника. Чеков нет, столовая не ведёт учёт продаж.
Пользователи (роли)	Студент — заказывает и оплачивает обед, забирает его по коду. Сотрудник столовой — собирает заказы и выдаёт их по коду. Администратор столовой — ведёт меню, остатки и лимиты окон, смотрит отчёт продаж.
Ценность	Студент заранее заказывает и оплачивает обед и забирает его без очереди в выбранную перемену. Столовая впервые получает учёт каждого онлайн-заказа.
Функции	10 функций первой версии.
Бизнес-модель	Комиссия 5 % с оплаченного онлайн-заказа после бесплатного пилотного месяца (раздел 1.3). Фиксированная плата 500 000 сум в месяц отклонена по интервью П5.
Конкуренты	Yandex Go «Еда» и Uzum Tezkor везут еду к человеку, но не дают предзаказа на выбранное время и не работают со столовой внутри университета.
Границы	Одна точка выдачи на цокольном этаже. Без доставки, без своей кухни, без программы лояльности в первой версии (US-20, Won’t).
Метрика	Среднее время получения предзаказа в большую перемену — не больше 2 минут через 3 месяца после запуска (задача BG-1).

Проблема описана через ситуацию у стойки, а не через отсутствие продукта. Тема 3 называет описание проблемы через отсутствие продукта первой из семи типичных ошибок, и One-Pager её не содержит. Числа ожидания в строке «Проблема» — оценка команды по личному опыту. Замеры раздела 2.3 дали среднее 5,4 минуты и максимум 8,5 минуты (модельные данные): оценка команды лежит между средним и максимумом.

Вывод. One-Pager задаёт роли, функции и метрику, на которые опираются главы 2 и 3. Любое изменение этих строк команда записывает в приложение И.

1.2.1. Обоснование выбора идеи и её риски

Команда выбрала идею А по трём причинам. Первая причина — проблему видно без продукта. Члены команды сами стоят в очереди в большую перемену, а двое из трёх опрошенных однокурсников не успевали поесть. Идея В решает проблему, о которой команда слышала меньше: студенты находят аудитории по расписанию, и ни один респондент опроса 09.10 не назвал навигацию трудностью. Идея С полезна, но у неё нет денежного потока: планировщик сроков можно продавать только по подписке или с рекламой, а реклама без расчёта — одна из семи ошибок темы 3.

У идеи А три риска. Первый риск — столовая может отказаться от сервиса: без её согласия продукта нет. Команда снижает этот риск бесплатным пилотным месяцем и комиссией только с продаж (раздел 1.3). Второй риск — студенты могут не захотеть платить заранее. Интервью П3 показало страх потерять деньги (модельные данные), поэтому в бэклог добавлена отмена заказа с возвратом денег (US-08). Третий риск — экономика одной столовой. Расчёт раздела 1.3 показывает, что одна столовая не покрывает постоянные расходы сервиса. Поэтому идея жизнеспособна как пилот в одной столовой, а как бизнес — только при трёх столовых и больше.

1.3. Бизнес-модель и юнит-экономика

1.3.1. Холст бизнес-модели

Холст бизнес-модели собирает модель продукта на одном листе из девяти блоков (рисунок 1.1). Номера блоков показывают порядок заполнения: от сегментов клиентов к структуре издержек. Сегменты холста совпадают с тремя ролями One-Pager (таблица 1.1).

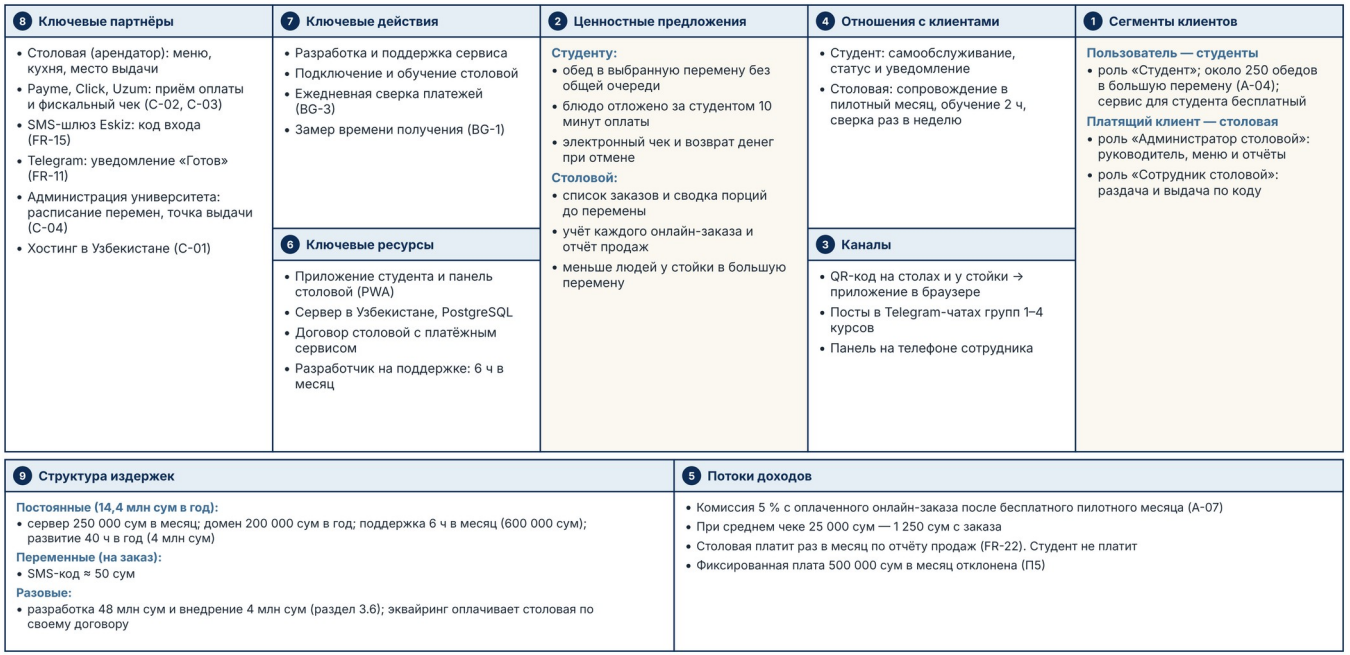


Рисунок 1.1 — Холст бизнес-модели сервиса предзаказа

Холст показывает двустороннюю модель. Пользователь — студент, он не платит за сервис. Платящий клиент — столовая; в системе её представляют две роли: «Администратор столовой» и «Сотрудник столовой». Поток доходов один: комиссия 5 % с оплаченного онлайн-заказа после бесплатного пилотного месяца. Издержки разделены на три вида. Постоянные издержки — 14,4 млн сум в год: сервер, домен, поддержка и развитие. Переменные издержки — около 50 сум на заказ за SMS-код входа. Разовые издержки — разработка за 48 млн сум и внедрение в первой столовой за 4 млн сум (раздел 3.6). Эквайринг не входит в наши издержки: платёж идёт на счёт столовой по её договору с платёжным сервисом (ограничение C-03).

Вывод. Модель зависит от одного партнёра — столовой. Поэтому блок «Отношения с клиентами» содержит сопровождение пилотного месяца, а отчёт продаж (US-18) входит в первую версию.

1.3.2. Холст ценностного предложения

Тема 3 требует, чтобы у каждой функции One-Pager была пара в профиле клиента: боль, которую функция снимает, или выгода, которую она даёт. Команда проверила все 10 функций. Пять функций студента из семи имеют пару: меню с остатком снимает страх, что блюдо закончится, оплата заранее — очередь у кассы, выбор перемены и код — очередь у стойки. Три функции столовой тоже имеют пары: панель выдачи, меню с остатками и отчёт продаж закрывают хаос у стойки, подсчёт «на глаз» и отсутствие учёта.

Две функции пары не имеют. Уведомление о готовности только экономит одно действие, потому что статус заказа уже показывает, когда идти за обедом. История и повтор заказа не связаны ни с одной болью студента. Обе функции команда перенесла в список «может быть»: они остались в бэклоге с приоритетами Should и Could, а не Must (раздел 2.6).

1.3.3. Юнит-экономика

Юнит-экономика считает доход и расходы на одну единицу продажи. Единица продажи сервиса — один оплаченный онлайн-заказ. Таблица 1.2 считает экономику одной столовой при целевом объёме задачи BG-2 и при объёме на 30 % ниже целевого. Для каждого числа указан источник или допущение.

Таблица 1.2 — Юнит-экономика одной столовой

Показатель	Цель	Спрос −30 %	Источник или допущение
Онлайн-заказов в учебный день	50	35	BG-2: 20 % от 250 обедов большой перемены (А-04, П4 — модельные данные)
Заказов за учебный месяц	1 100	770	22 учебных дня в месяце
Средний чек	25 000 сум	25 000 сум	Допущение: блюдо и напиток по ценам прототипа (экран Э-1)
Доход сервиса с заказа	1 250 сум	1 250 сум	Комиссия 5 % (А-07, П5 — модельные данные)
Переменные расходы на заказ	50 сум	50 сум	SMS-код входа: около 500 SMS в месяц по 100 сум на 1 100 заказов (раздел 3.6)
Маржа с заказа	1 200 сум	1 200 сум	Доход минус переменные расходы
Маржа за учебный месяц	1 320 000 сум	924 000 сум	Маржа с заказа × заказы
Постоянные расходы на учебный месяц	1 600 000 сум	1 600 000 сум	14,4 млн сум в год / 9 учебных месяцев (таблица 3.4)
Результат за учебный месяц	−280 000 сум	−676 000 сум	Маржа минус постоянные расходы
Точка безубыточности	61 заказ в день	61 заказ в день	1 600 000 / 1 200 / 22
Стоимость привлечения студента	2 200 сум	2 200 сум	330 000 сум на QR-наклейки (60 столов) и 10 плакатов / 150 студентов
Окупаемость привлечения студента	2 заказа	2 заказа	2 200 / 1 200 = 1,8 заказа
Стоимость привлечения столовой	1 575 000 сум	1 162 500 сум	Пилотный месяц без комиссии + обучение 2 ч по 100 000 сум

Строки таблицы делятся на три группы. Первая группа — объём: 50 заказов в день взяты из задачи BG-2, а 35 заказов — проверка на 30 % ниже цели. Вторая группа — маржа: сервис получает 1 250 сум с заказа и тратит около 50 сум на SMS, поэтому маржа равна

1 200 сум. Третья группа — постоянные расходы. Сервер и поддержка работают 12 месяцев в году, а заказы идут только 9 учебных месяцев, поэтому годовые 14,4 млн сум делятся на 9, а не на 12.

Вывод. Одна столовая не окупает сервис даже при целевом объёме. Модель становится жизнеспособной при нескольких столовых: при трёх столовых маржа покрывает постоянные расходы.

1.3.4. Разбор юнит-экономики

Каждое число таблицы 1.2 имеет источник. Объём 250 обедов в большую перемену назвал сотрудник раздачи в интервью П4, а долю 20 % задаёт задача BG-2. Комиссию 5 % и бесплатный пилотный месяц назвал руководитель столовой в интервью П5. Интервью П4 и П5 — модельные данные: команда не получила доступ к сотрудникам столовой и составила ответы по своему опыту. Поэтому эти два числа — главные допущения расчёта, и пилот должен проверить их первыми.

При спросе на 30 % ниже цели заказов становится 35 в день. Доход с заказа не меняется, а маржа за месяц падает с 1 320 000 до 924 000 сум. Постоянные расходы не падают, поэтому убыток растёт с 280 000 до 676 000 сум за учебный месяц. Падение спроса не разрушает модель трёх столовых: 105 заказов в день дают 2 772 000 сум маржи против 2 133 000 сум постоянных расходов. Но тогда вложения в разработку и внедрение (52 млн сум) возвращаются примерно за 9 лет. При целевом спросе три столовые возвращают вложения чуть больше чем за 3 года, а при спросе на 30 % ниже цели для возврата за 3 года нужно 5 столовых.

1.4. Устав проекта и модель жизненного цикла

Цель проекта — создать сервис предзаказа и оплаты обеда в университетской столовой с получением в выбранное окно перемены. Сервис убирает очередь в часы пик и даёт столовой учёт заказов.

По замечанию преподавателя на защите этапа 2 бизнес-цели записаны как одна общая цель и четыре задачи. Таблица 1.3 показывает цель BG-0 и задачи BG-1…BG-4. Каждая задача измерима и имеет срок (SMART).

Таблица 1.3 — Бизнес-цель BG-0 и её задачи

ID	Формулировка	Как измеряем
BG-0	**Цель.** За 3 месяца после запуска студент получает обед в большую перемену без очереди, а столовая ведёт учёт каждого онлайн-заказа.	Через задачи BG-1…BG-4
BG-1	Сократить среднее время получения заказа через приложение в большую перемену до 2 минут через 3 месяца после запуска. Сейчас, по оценке команды, это 3–7 минут.	Отметки «Я на месте» и «Выдан» в панели столовой плюс контрольные замеры секундомером (5 перемен)
BG-2	Довести долю заказов через приложение до 20 % от всех обедов в большую перемену через 3 месяца после запуска.	Число выданных онлайн-заказов к числу всех выдач за день (данные столовой)
BG-3	Сохранять в системе 100 % оплаченных онлайн-заказов с первого дня пилота. Через 1 месяц пилота столовая получает отчёт продаж без ручного подсчёта.	Сверка отчёта с выпиской платёжного сервиса за день: 0 расхождений
BG-4	Снизить долю онлайн-заказов, которые столовая отменяет из-за того, что блюдо закончилось, до 3 % и ниже в первые 3 месяца.	Число отмен по причине «нет блюда» к числу всех онлайн-заказов за месяц

Цель BG-0 соединяет две ценности One-Pager: обед без очереди для студента и учёт для столовой. Задачи делят цель на проверяемые части. Задача BG-1 измеряет время получения: сейчас 3–7 минут по оценке команды, цель — 2 минуты. Задача BG-2 измеряет долю онлайн-заказов; 20 % — объём, на котором построена юнит-экономика раздела 1.3. Задача BG-3 измеряет учёт: 100 % оплаченных заказов в системе. Задача BG-4 добавлена на этапе 2 по гипотезе H3: блюдо не должно заканчиваться после оплаты заказа. Номера BG-1…BG-4 не изменились с этапа 2, поэтому связи в матрице трассируемости (таблица 2.2) остались верными.

Вывод. Цель и задачи — вершина трассировки. Каждое требование главы 2 и каждое решение главы 3 ссылается на одну из задач BG-1…BG-4.

1.4.1. Обоснование модели жизненного цикла

Каскадная модель проиграла по двум главным критериям. Первый критерий — стабильность требований. Каскадная модель фиксирует требования в начале, а за этап 2 в продукте изменились 9 пунктов (раздел 2.3). В каскадной модели каждое такое изменение требовало бы нового согласования всей спецификации. Второй критерий — скорость первого результата. Каскадная модель даёт работающий результат в конце проекта, а команде нужно было показать сценарий студентам на этапе 3, до разработки. Третий критерий — форма договора: у команды нет договора с фиксированной ценой, поэтому главное преимущество каскадной модели не работает.

Scrum в чистом виде тоже проиграл по одному критерию — цене ошибки. В платёжном контуре (FR-07, FR-08, FR-09, FR-14) ошибку нельзя отложить до следующего спринта: студент уже потерял деньги. Поэтому для этого контура команда берёт принцип V-модели: требования и тест-кейсы фиксируются до разработки. Тест-кейсы TC-013…TC-017 и TC-028…TC-031 записаны уже в главе 2, а диаграмма последовательности оплаты (рисунок 3.5) задаёт порядок их проверки. Недоступность заказчика команда закрывает ролью Product Owner: участник А представляет интересы столовой между встречами, а с руководителем столовой команда встречается раз в итерацию.

Выводы по главе 1

Проблема очереди подтверждена реальными данными малой выборки. Двое из трёх однокурсников не успевали поесть из-за очереди, а ожидание в толпе, по оценке команды, занимает 6–7 минут из 10-минутной малой перемены. Прямого конкурента с предзаказом на время внутри университета нет. Модельный опрос этапа 2 даёт тот же вывод: 23 студента из 36 хотя бы раз за 2 недели ушли без обеда.

Бизнес-модель — комиссия 5 % со столовой. Одна столовая даёт убыток 280 000 сум за учебный месяц, точка безубыточности — 61 заказ в день. Поэтому идея проходит фильтр жизнеспособности только при трёх столовых и больше, а пилот в одной столовой проверяет спрос. При спросе на 30 % ниже цели убыток одной столовой растёт до 676 000 сум в месяц. Сегменты холста совпадают с тремя ролями One-Pager, а две функции без пары в профиле клиента — уведомление и повтор заказа — перенесены в список «может быть».

Проект ведётся по гибриду Scrum и V-модели, потому что требования меняются, а ошибка в оплате стоит денег студента. Глава 2 получает из главы 1 цель BG-0, три роли и десять функций, по которым команда собирает требования.

Глава 2. Требования

Глава 1 показала, что проблема очереди существует и что бизнес-модель работает при трёх столовых и больше. Глава 2 отвечает на следующий вопрос: что именно должна делать система, чтобы студент получил обед без очереди, а столовая получила учёт каждого заказа. Это следующий шаг после главы 1, потому что без проверяемых требований нельзя построить модели процессов, данных и прототип главы 3. Глава содержит видение и границы продукта, анализ заинтересованных сторон, результаты исследования, персоны, спецификацию требований версии 2.1, бэклог с приоритетами и матрицу трассируемости.

2.1. Видение и границы продукта (Vision & Scope)

Для студентов университета, которым не хватает перемены на обед, сервис предзаказа даёт заказать и оплатить обед заранее и забрать его по коду в выбранную перемену. В отличие от Yandex Go и Uzum Tezkor, сервис работает со столовой внутри университета и привязан к расписанию перемен. Для столовой сервис даёт список заказов по переменам и учёт каждого оплаченного заказа. Цель продукта BG-0 и задачи BG-1…BG-4 даны в таблице 1.3.

Первая версия покрывает один сценарий: заказ, оплата и получение в одной столовой. В неё входят меню с остатком, корзина и выбор перемены, оплата через Payme, Click и Uzum, статус и код получения, панель столовой и отчёт продаж. В первую версию не входят доставка, оплата наличными через приложение, программа лояльности (US-20, Won’t), оценки блюд и другие столовые. Версия 2 подключает другие столовые и кафе рядом с университетом, версия 3 — лояльность и заказ для преподавателей. Порядок версий следует из юнит-экономики раздела 1.3: одна столовая не покрывает постоянные расходы, а три столовые покрывают.

Раздел 2.1 фиксирует, зачем продукт нужен бизнесу и что входит в первую версию. Документ Vision & Scope — основа спецификации: требования разделов 2.5 и 2.6 не выходят за его границы.

2.2. Заинтересованные стороны

Команда выделила восемь заинтересованных сторон и разместила их на матрице «влияние — интерес». Управлять плотно нужно тремя сторонами: руководством столовой, сотрудниками столовой и преподавателем курса. Руководство решает, будет ли пилот, а сотрудники выдают заказы и первыми видят ошибки сервиса; с ними команда проводит интервью П4 и П5 и согласует спецификацию. Студенты — главные пользователи с высоким интересом и средним влиянием: их держат в курсе через опрос и проверку прототипа. Платёжные сервисы Payme, Click и Uzum имеют высокое влияние, потому что без их API нет оплаты, поэтому команда учитывает их правила как ограничение С-02. Полная матрица и таблица сторон даны в полной версии работы.

2.3. Исследование: интервью, опрос и замеры

2.3.1. Результаты исследования

Исследование подтвердило главную проблему, но изменило её форму. На этапе 1 команда считала, что студент ждёт обед в большую перемену. Модельные замеры дали в среднем 5,4 минуты, медиану 5,0 минуты и максимум 8,5 минуты на 25 студентах. Эти числа лежат внутри реального диапазона опроса 09.10 (от 2–3 до 5–10 минут) и внутри оценки команды (3–7 минут). Поэтому базовое значение для задачи BG-1 команда

оставила «3–7 минут», а цель — 2 минуты. Новый факт дал вопрос В3: 31 % студентов в последний раз пришли в столовую в малую перемену, а не в большую. Интервью П2 объясняет причину: студентка приходит в первую перемену, «чтобы что-то досталось». Значит, очередь гонит студентов в малые перемены, где времени ещё меньше. Поэтому окна получения в требовании FR-04 включают все перемены дня, а не только большую.

Опровергнута одна гипотеза — Н6 в исходной форме. Команда считала, что у стойки есть планшет или компьютер с интернетом. Интервью П4 показало другое: у стойки нет Wi-Fi и планшета, а у сотрудника есть свой телефон Android с мобильным интернетом. Это расхождение изменило три требования. Панель столовой работает в браузере телефона сотрудника (NFR-10). Ввод кода идёт с крупной цифровой клавиатуры, потому что руки сотрудника заняты подносом (NFR-09). Допущение А-06 получило статус «изменено». Также опровергнут вариант бизнес-модели с фиксированной платой: в интервью П5 руководитель столовой согласился на процент с заказа, но не на фиксированный платёж. Поэтому бизнес-модель раздела 1.3 использует комиссию 5 % после бесплатного пилотного месяца (А-07).

Главное ограничение этих выводов — модельные данные. Модельные данные показывают, как команда понимает проблему, но не доказывают её. Поэтому команда отделила решения, которые опираются на реальные данные, от решений, которые опираются только на модельные данные. Проблема очереди и нехватки блюд опирается на реальный опрос 09.10 и на оценки команды. Числа 250 обедов, лимит 40 заказов, комиссия 5 % и доли опроса — только на модельные данные. Эти числа пилот проверит первыми, и при расхождении команда изменит требования через историю изменений (приложение И).

2.4. Персоны и карта пути

Персоны описывают три роли продукта обобщённо, без имён и возраста сотрудников. Студент обедает в столовой 2–3 раза в неделю, в большую перемену стоит в очереди и иногда уходит без обеда; ему нужно знать заранее, есть ли блюдо, и не терять деньги, если планы изменятся. Сотрудник столовой работает у стойки с личным телефоном Android и без Wi-Fi; ему нужен крупный код и выдача за несколько секунд. Администратор столовой считает остатки «на глаз» и не видит продаж за день; ему нужны меню с остатками и отчёт. Карта пути студента показала два провала эмоций: на этапе «Потребность», когда перемена длится 10 минут, а очередь занимает 6–7 минут, и на этапе «Заказ и оплата», где студент боится потерять деньги. Эти провалы дали требования резерва (FR-09), отмены с возвратом (FR-13, FR-14) и экрана ошибки оплаты в прототипе (раздел 3.5).

2.5. Спецификация требований (SRS), версия 2.1

2.5.1. Функциональные требования

Спецификация содержит 27 функциональных требований FR-01…FR-27; полный список с источником каждого требования дан в приложении В. Требования делятся на группы по ролям. Студенту служат FR-01…FR-15, FR-24, FR-25 и FR-27: меню с остатком, корзина, выбор перемены, оплата, резерв на 10 минут (FR-09), статус, код получения (FR-12), отмена с возвратом, вход по SMS, история заказов и отметка «Я на месте». Сотруднику столовой служат FR-16…FR-19: заказы по окнам, смена статуса, выдача по коду и стоп-лист. Администратору служат FR-20…FR-22 и FR-26: меню, лимит окна, отчёт продаж и время получения по окнам. Требование FR-23 сохраняет каждый оплаченный заказ для учёта (BG-3). Каждое требование имеет источник — интервью, опрос или ограничение — и связано с историей бэклога.

2.5.2. Нефункциональные требования

Нефункциональное требование описывает, насколько хорошо система делает свою работу. В версии 2.1 каждое нефункциональное требование записано методом пяти шагов темы 5. Шаг 1 — выбрать характеристику качества по стандарту ISO/IEC 25010. Шаг 2 — назвать меру, то есть что именно измеряем. Шаг 3 — задать порог с числом. Шаг 4 — задать условия, при которых порог должен выполняться. Шаг 5 — назвать способ проверки. Итоговая формулировка соединяет пять шагов в одно предложение. На этапе 2 таблица уже имела столбцы шагов, но формулировка часто не содержала порога и условий, например «Система доступна в учебные дни с 07:30 до 17:00». В версии 2.1 порог и условия стоят в самой формулировке.

Таблица 2.1 показывает 10 нефункциональных требований версии 2.1. Столбцы 1–5 соответствуют пяти шагам метода. Столбец BG связывает требование с задачей бизнес-цели, а столбец TC — с тест-кейсом этапа 4.

Таблица 2.1 — Нефункциональные требования версии 2.1 (метод пяти шагов)

ID	Требование (версия 2.1)	1. Характеристика ISO/IEC 25010	3. Порог	5. Проверка	BG	TC
NFR-01	Меню дня открывается на смартфоне студента не дольше 2 с для 95 % запросов при 100 одновременных пользователях в сети 4G.	Производительность — временные характеристики	≤ 2 с для 95 % запросов	Нагрузочный тест (k6), 10 минут	BG-1	TC-101
NFR-02	Оплаченный заказ получает статус «Принят» не позднее 5 с после подтверждения оплаты для 95 % заказов при потоке 60 заказов за 10 минут.	Производительность — временные характеристики	≤ 5 с для 95 % заказов	Нагрузочный тест с тестовым шлюзом	BG-1	TC-102
NFR-03	Система отвечает на запросы не меньше 99 % времени учебных дней с 07:30 до 17:00 за месяц: простой не больше 2 ч в месяц.	Надёжность — доступность	≥ 99 % в месяц (простой ≤ 2 ч)	Мониторинг (UptimeRobot), отчёт за месяц	BG-2	TC-103
NFR-04	После сбоя сервера во время оплаты ни один оплаченный заказ не теряется: 0 расхождений между выпиской платёжного сервиса и отчётом за день.	Надёжность — восстанавливаемость	0 заказов	Тест с остановкой сервера; ежедневная сверка	BG-3	TC-104
NFR-05	Номер банковской карты не попадает на сервер системы: 0 полей с номером карты в базе данных и логах во всех сценариях оплаты.	Безопасность — конфиденциальность	0 полей	Инспекция базы данных и логов	BG-3	TC-105
NFR-06	Все страницы и методы API работают только по HTTPS с TLS 1.2 и выше; оценка SSL Labs — A и выше.	Безопасность — целостность	TLS 1.2 и выше; оценка A	Проверка SSL Labs	BG-3	TC-106
NFR-07	Панель столовой отклоняет 100 % запросов без входа сотрудника; сессия сотрудника длится не больше 12 ч.	Безопасность — подлинность	Отказ 100 %; сессия ≤ 12 ч	Тест доступа без входа и после 12 ч	BG-3	TC-107
NFR-08	4 из 5 новых студентов оформляют первый заказ без помощи не дольше 90 с от открытия меню до оплаты.	Удобство — изучаемость	≤ 90 с у 4 из 5 студентов	Юзабилити-тест прототипа, 5 студентов	BG-2	TC-108
NFR-09	Сотрудник выдаёт заказ не дольше 10 с от ввода кода до отметки «Выдан» в большую перемену у стойки.	Удобство — управляемость	≤ 10 с	Секундомер на прогоне прототипа, 10 выдач	BG-1	TC-109

ID	Требование (версия 2.1)	1. Характеристика ISO/IEC 25010	3. Порог	5. Проверка	BG	TC
NFR-10	Сценарии US-01…US-07 работают без ошибок на 100 % проверенных устройств; панель столовой работает на телефоне Android сотрудника.	Совместимость — взаимодействие	100 %	Ручной прогон на 4 устройствах	BG-2	TC-110

Требования покрывают пять характеристик ISO/IEC 25010: производительность (NFR-01, NFR-02), надёжность (NFR-03, NFR-04), безопасность (NFR-05…NFR-07), удобство использования (NFR-08, NFR-09) и совместимость (NFR-10). Больше всего требований работают на задачу BG-3: четыре требования защищают оплату и учёт заказа. У каждого требования один порог с числом и один способ проверки, поэтому каждое требование получило один тест-кейс TC-101…TC-110.

Два требования уже проверены на прототипе. NFR-08 выполнено: 4 участника из 5 оформили заказ меньше чем за 90 с. NFR-09 выполнено: выдача заняла от 6 до 9 с. Это результаты проверки прототипа (раздел 3.5), и они тоже модельные данные. Остальные восемь требований проверят нагрузочный тест, мониторинг и инспекция кода на этапе 4.

Вывод. Все нефункциональные требования измеримы и проверяемы. В главе 3 каждое требование получило архитектурное решение (раздел 3.4).

Разбор нефункциональных требований. Пороги производительности взяты из расписания перемен и из объёма заказов. Малая перемена длится 10 минут, а заказ в перемену должен занимать меньше двух минут. Поэтому меню должно открываться не дольше 2 с (NFR-01): при пяти переходах между экранами студент теряет на ожидание не больше 10 с. Нагрузку 100 одновременных пользователей команда получила так: цель BG-2 даёт 50 заказов на большую перемену, и в пиковые 10 минут меню смотрят примерно вдвое больше студентов, чем потом заказывают. Порог NFR-02 (5 с до статуса «Принят» при 60 заказах за 10 минут) взят с запасом 20 % над целью BG-2. Оба требования проверяет нагрузочный тест в k6 на тестовом стенде с тестовым платёжным шлюзом (TC-101, TC-102).

2.6. Бэклог и приоритеты

2.6.1. Обоснование приоритетов MoSCoW

Группа Must содержит минимальную цепочку, без которой нет ни одного заказа, и истории главной ценности. Минимальная цепочка — меню (US-01), корзина (US-02), оплата (US-04), вход по телефону (US-11) и публикация меню администратором (US-16). Если не сделать любую из этих историй, студент не сможет оформить заказ, а задача BG-2 (20 % онлайн-заказов) получит значение 0. Например, без входа по телефону (US-11) система не свяжет заказ со студентом, и студент не найдёт свой код получения.

Вторая часть группы Must — истории, которые дают обед без очереди: окно получения (US-03), статус заказа (US-05), выдача по коду (US-07), список заказов по окнам (US-12), смена статуса (US-13) и поиск заказа по коду (US-14). Без них онлайн-заказ возвращается в общую очередь. Без окна получения все онлайн-заказы придут в большую перемену, и кухня не соберёт их заранее. Без статуса студент придёт раньше и встанет у стойки. Без поиска по коду сотрудник будет спрашивать имя вслух, а система не запишет время выдачи, и задачу BG-1 нельзя будет измерить.

2.7. Матрица трассируемости

Таблица 2.2 — главная таблица связи глав 1, 2 и 3. Одна строка таблицы — одна история. Варианты использования взяты с рисунка 3.3, экраны — из раздела 3.5, таблицы базы данных — из раздела 3.3.

Таблица 2.2 — Матрица трассируемости: BG → US → FR → UC → экран → таблицы БД → ТС

US	Приоритет	BG	FR	UC	Экран	Таблицы БД	ТС
US-01	Must	BG-2, BG-4	FR-01, FR-02	UC-02	Э-1	daily_menus, menu_items, dishes	TC-001…004
US-02	Must	BG-2	FR-03	UC-03	Э-1, Э-2	menu_items	TC-005…008
US-03	Must	BG-1	FR-04, FR-05, FR-06	UC-05, UC-06	Э-3	pickup_windows	TC-009…012
US-04	Must	BG-2, BG-3, BG-4	FR-07, FR-08, FR-09, FR-23	UC-05, UC-07	Э-4, Э-5	reservations, reservation_items, payments, orders, order_items	TC-013…017
US-05	Must	BG-1	FR-10	UC-10	Э-6	orders, order_status_log	TC-018…020
US-06	Should	BG-1	FR-11	UC-11	Telegram	students	TC-021…023
US-07	Must	BG-1	FR-12, FR-27	UC-12	Э-6	orders	TC-024…027
US-08	Should	BG-2, BG-3	FR-13, FR-14	UC-08, UC-09	Э-6	refunds, payments	TC-028…031
US-09	Could	BG-2	FR-25	UC-04	—	orders, order_items	TC-032…034
US-10	Could	BG-2	FR-24	UC-13	—	orders	TC-035…037
US-11	Must	BG-2	FR-15	UC-01	—	students	TC-038…041
US-12	Must	BG-1	FR-16	UC-14	Э-7	orders, pickup_windows	TC-042…045
US-13	Must	BG-1	FR-17	UC-15	—	orders, order_status_log	TC-046…049
US-14	Must	BG-1, BG-3	FR-18	UC-16	Э-7	orders	TC-050…053
US-15	Should	BG-4	FR-19	UC-17	—	menu_items	TC-054…056
US-16	Must	BG-2, BG-4	FR-20	UC-18	—	daily_menus, menu_items, dishes	TC-057…060
US-17	Should	BG-1	FR-21	UC-19	—	pickup_windows	TC-061…063
US-18	Should	BG-3	FR-22	UC-20	—	orders, order_items, payments	TC-064…067
US-19	Could	BG-1	FR-26	UC-21	—	orders, order_status_log	TC-068…070
US-20	Won’t	—	—	—	—	—	—

Все 19 историй первой версии имеют задачу BG, требование, вариант использования, таблицу базы данных и тест-кейсы. Все 27 функциональных требований стоят хотя бы в одной строке, поэтому требований без истории нет. Девять историй имеют экран прототипа: это все истории главного пути студента и главные истории сотрудника. История US-06 работает через Telegram, поэтому у неё нет экрана. Истории администратора и вспомогательные истории студента пока не имеют экрана; их экраны команда нарисует на этапе 4.

Матрица работает в обе стороны. Прямой путь: задача BG-1 → история US-03 → требование FR-05 → вариант использования UC-06 → экран Э-3 → таблица pickup_windows → тест-кейс TC-010. Обратный путь: тест-кейс TC-010 проверяет критерий 2 истории US-03, критерий опирается на требование FR-05, требование работает на задачу BG-1, а задача опирается на гипотезу H1 и на реальный опрос 09.10. История US-20 (Won’t) не имеет связей, а требование FR-28 удалено, потому что у него нет задачи BG.

Вывод. Трассировка не прерывается от задачи бизнес-цели до тест-кейса. Пробелы матрицы — экраны администратора и входа — записаны в план этапа 4.

Выводы по главе 2

Исследование подтвердило проблему очереди и нехватки блюд и изменило продукт. Модельные замеры (5,4 минуты в среднем) совпали с реальным опросом 09.10 и с оценкой команды. Опровергнутая гипотеза H6 перенесла панель столовой на телефон сотрудника, а нехватка блюд у 27 студентов из 36 дала задачу BG-4 и резерв FR-09. Выводы, которые опираются только на модельные данные, отмечены в тексте, а план их проверки дан в заключении.

Спецификация версии 2.1 содержит 27 функциональных и 10 нефункциональных требований. Каждое нефункциональное требование записано методом пяти шагов, имеет порог с числом, условия, способ проверки, задачу BG и тест-кейс. В требованиях нет слов «быстро», «удобно», «надёжно» и «по возможности» без чисел. Проверка по семи признакам нашла шесть ошибок, и все шесть исправлены в версии 2.1.

Матрица трассируемости связывает каждую историю с задачей BG, требованием, вариантом использования, таблицей базы данных и тест-кейсом. Глава 3 получает из главы 2 три роли, 21 вариант использования и требования, по которым команда строит процессы, модели, схему данных и прототип. У каждой из 19 историй первой версии есть не меньше трёх тест-кейсов, поэтому этап 4 начнёт проверку с готового списка.

Глава 3. Проектирование и выбор платформы

Глава 2 определила, что должна делать система: 27 функциональных и 10 нефункциональных требований, 19 историй первой версии и три роли. Глава 3 отвечает на вопрос, как система выполняет эти требования и на какой платформе её строить. Это следующий шаг после главы 2, потому что модели процесса, данных и архитектуры можно строить только по утверждённой спецификации. Глава содержит модели процесса «как есть» и «как будет», UML-модели, схему базы данных, архитектуру, прототип интерфейса и выбор платформы со стоимостью владения за 3 года.

3.1. Бизнес-процессы

3.1.1. Процесс «как есть»

Модель «как есть» описывает текущий процесс большой перемены по интервью П1–П5, опросу и замерам главы 2. Рисунок 3.1 показывает процесс от звонка до обеда или до ухода без обеда. Номера в кругах отмечают пять проблем процесса.

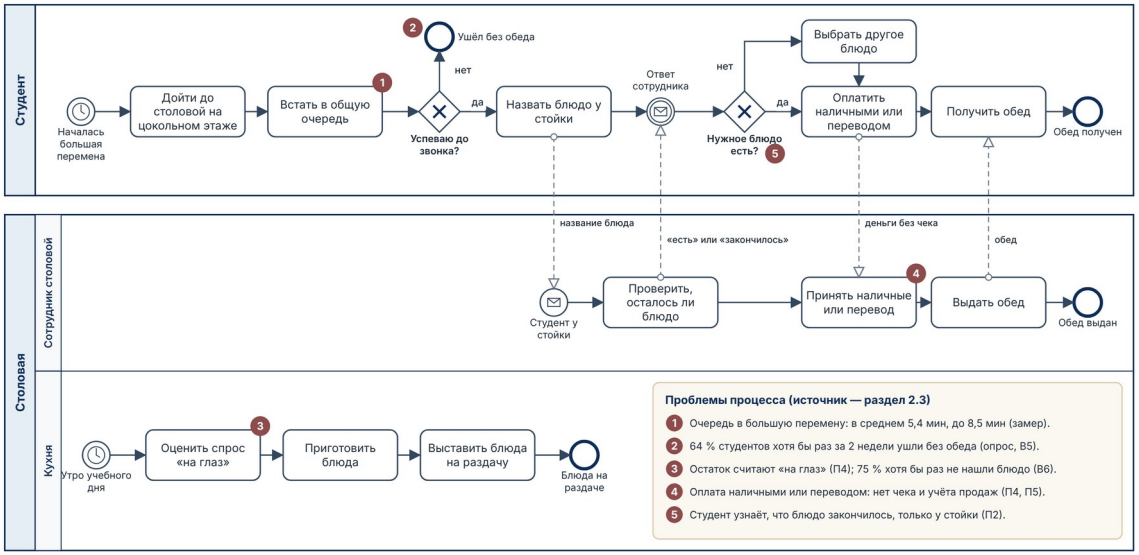


Рисунок 3.1 — Процесс получения обеда «как есть» (BPMN 2.0)

В процессе два пула: «Студент» и «Столовая» с дорожками «Сотрудник столовой» и «Кухня». Студент начинает процесс по таймеру «Началась большая перемена». Шлюз «Успеваю до звонка?» имеет ветвь «нет», которая заканчивается событием «Ушёл без обеда». Эту ветвь подтвердили двое из трёх реальных респондентов 09.10 и 23 студента из 36 в модельном опросе (В5). Шлюз «Нужное блюдо есть?»

стоит после очереди, поэтому студент узнаёт о нехватке блюда только у стойки. Кухня оценивает спрос «на глаз», а оплата идёт наличными или переводом на карту сотрудника без чека.

Вывод. Процесс «как есть» имеет пять проблем: общая очередь, уход без обеда, нехватка блюда, оплата без чека и неизвестный остаток. Процесс «как будет» должен убрать каждую из них.

3.1.2. Процесс «как будет»

Рисунок 3.2 показывает часть 1: от публикации меню до оплаченного заказа. В этой части работают четыре участника, и один из них — внешний платёжный сервис.

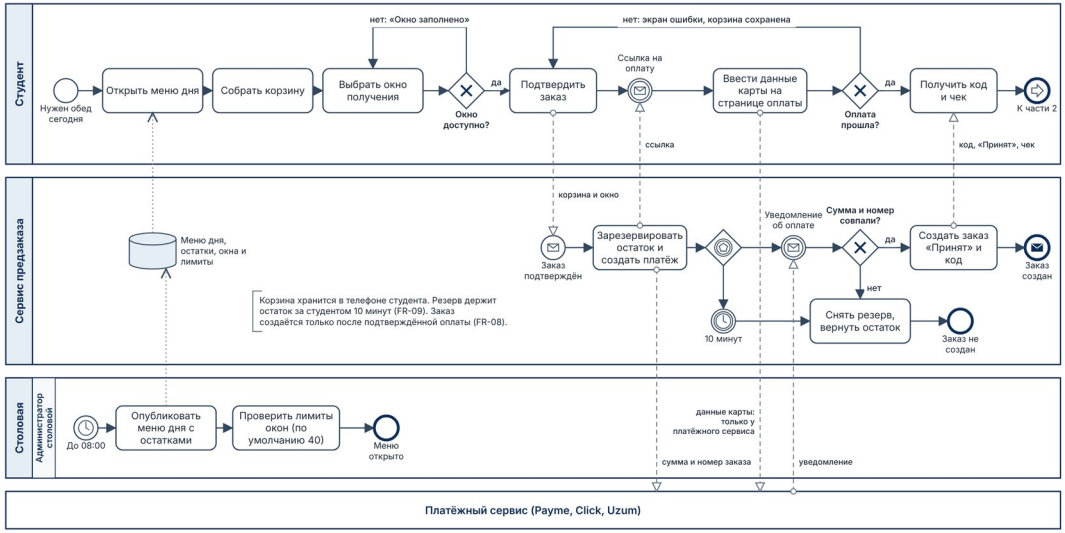


Рисунок 3.2 — Процесс «как будет», часть 1: заказ и оплата (BPMN 2.0)

В части 1 четыре пула: «Студент», «Сервис предзаказа», «Столовая» с дорожкой «Администратор столовой» и свёрнутый пул «Платёжный сервис». Администратор публикует меню дня с остатками до 08:00 и задаёт лимиты окон. Если окно заполнено, шлюз возвращает студента к выбору окна (FR-06). Сервис резервирует остаток и создаёт платёж, а шлюз на основе событий ждёт одно из двух событий: уведомление об оплате или таймер «10 минут». Если таймер срабатывает первым, сервис снимает резерв и возвращает остаток (FR-09). Заказ появляется только после проверки суммы и номера платежа (FR-08), а данные карты студент вводит только на странице платёжного сервиса (NFR-05).

Вывод. Часть 1 убирает проблему неизвестного остатка и оплаты без чека: студент видит остаток до заказа, резерв держит порцию на время оплаты, а чек формирует платёжный сервис.

3.1.3. Отличия процесса «как будет» от процесса «как есть»

Таблица 3.1 сравнивает процессы в формате «было — стало — что даёт». Каждая строка связана с номером проблемы на рисунке 3.1 и с задачей BG. Данные столбца «Было» взяты из главы 2.

Таблица 3.1 — Отличия процесса «как будет» от процесса «как есть»

№	Было («как есть»)	Стало («как будет»)	Что даёт	BG
1	Студент стоит в общей очереди: в среднем 5,4 мин, максимум 8,5 мин (замеры, модельные данные). Проблема 1.	Студент заранее выбирает окно получения и забирает заказ по коду у отдельной точки выдачи.	Время получения — до 2 минут.	BG-1
2	Блюдо заканчивается: 27 из 36 студентов хотя бы раз не нашли нужное блюдо (В6). Кухня считает остаток «на глаз» (П4). Проблема 3.	Меню с остатками публикуется до 08:00. Резерв держит остаток 10 минут. Сотрудник отмечает «Закончилось» одним действием.	Студент видит только блюда, которые есть. Меньше отмен «нет блюда».	BG-4
3	Оплата наличными или переводом на карту сотрудника, без чека (П4, П5). Проблема 4.	Оплата через Payme, Click или Uzum. Чек формирует платёжный сервис. Каждый заказ хранится в базе.	100 % оплаченных заказов в учёте. Отчёт продаж без ручного подсчёта.	BG-3
4	Кухня не знает спрос заранее и готовит «на глаз». Проблема 3.	Заказы сгруппированы по окнам. Сотрудник видит сводку порций. Лимит окна — 40 заказов.	Кухня собирает заказы до начала перемены.	BG-1
5	Студент узнаёт о наличии и готовности только у стойки (П2). Проблема 5.	Статус заказа виден в приложении. Уведомление «Готов» приходит в Telegram.	Студент приходит, когда заказ готов.	BG-1
6	23 из 36 студентов уходили без обеда хотя бы раз за 2 недели (В5). Проблема 2.	Предзаказ на любую перемену дня. Отмена с возвратом денег до статуса «Готовится».	Больше обедов через приложение, меньше риска потерять деньги.	BG-2

Главные отличия — первые три строки. В первой строке очередь заменяют окно получения и код, поэтому время получения зависит от сотрудника, а не от длины очереди. Во второй строке остаток виден до заказа и держится резервом, поэтому студент не платит за блюдо, которого нет. В третьей строке оплата идёт через лицензированный платёжный сервис, поэтому у столовой появляется учёт и чек (С-02, С-03). Строки 4–6 поддерживают эти отличия: кухня знает спрос, студент знает статус, а отмена снижает риск предоплаты. Каждая из пяти проблем рисунка 3.1 закрыта хотя бы одной строкой.

Вывод. Сервис меняет 6 шагов процесса, и каждое изменение работает на одну задачу BG-1…BG-4. Раздел 3.1.4 разбирает, кому эти изменения дают пользу.

3.1.4. Разбор изменений процесса

Больше всех выигрывает студент. В процессе «как есть» студент тратит на очередь в среднем 5,4 минуты и узнаёт о нехватке блюда только у стойки. В процессе «как будет» студент выбирает блюдо и перемену на паре, платит за 90 секунд и забирает заказ у отдельной точки выдачи по коду. Целевое время получения — 2 минуты (BG-1), поэтому в большую перемену у студента остаётся больше времени на обед. Для студента, который приходит в малую перемену (31 % по В3), разница больше: 10-минутная перемена больше не уходит на очередь. Отмена с возвратом снимает страх предоплаты, который назвал респондент П3.

Сотруднику столовой становится легче в перемену, но у него появляется работа до перемены. В процессе «как есть» сотрудник в большую перемену один принимает заказы, считает деньги и слушает, как все кричат заказ одновременно (П4). В процессе «как будет» сотрудник собирает онлайн-заказы до звонка по сводке порций и выдаёт их по коду за 10 с и меньше (NFR-09). Деньги онлайн-заказов он не считает, потому что оплату проводит платёжный сервис. Новая работа сотрудника — смотреть панель, отмечать «Готов» и ставить блюдо в стоп-лист. Поэтому в договоре о пилоте команда записывает обучение сотрудника (2 часа, раздел 1.3).

3.2. UML-модели

3.2.1. Диаграмма вариантов использования

Диаграмма вариантов использования показывает, что делает с системой каждая роль. Рисунок 3.3 показывает три роли One-Pager, три внешние системы и 21 вариант использования. Верхняя часть границы системы — приложение студента, нижняя — панель столовой.

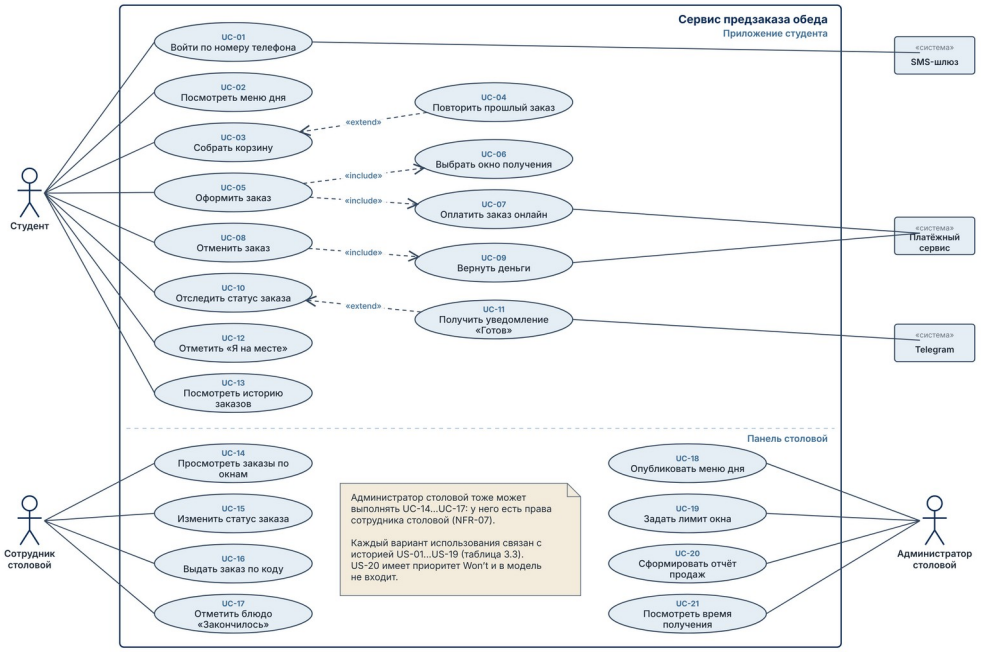


Рисунок 3.3 — Диаграмма вариантов использования (UML)

У студента 13 вариантов использования, у сотрудника столовой — 4, у администратора столовой — 4. Отношение «include» показывает обязательную часть: оформление заказа (UC-05) всегда включает выбор окна (UC-06) и оплату (UC-07), а отмена (UC-08) всегда включает возврат денег (UC-09). Отношение «extend» показывает необязательную часть: повтор прошлого заказа расширяет сборку корзины, а уведомление «Готов» расширяет отслеживание статуса. Внешние системы связаны только с теми вариантами, где они нужны: SMS-шлюз — со входом, платёжный сервис — с оплатой и возвратом, Telegram — с уведомлением. Заметка на диаграмме показывает, что администратор столовой имеет права сотрудника столовой (NFR-07).

Вывод. Все 19 историй первой версии покрыты вариантами использования. История US-20 имеет приоритет Won’t, поэтому у неё нет варианта использования.

3.2.2. Диаграммы деятельности

Диаграмма деятельности показывает шаги сценария с альтернативными ветвями. Рисунок 3.4 описывает варианты UC-05 «Оформить заказ» и UC-07 «Оплатить заказ онлайн». Дорожки показывают, кто выполняет каждый шаг.

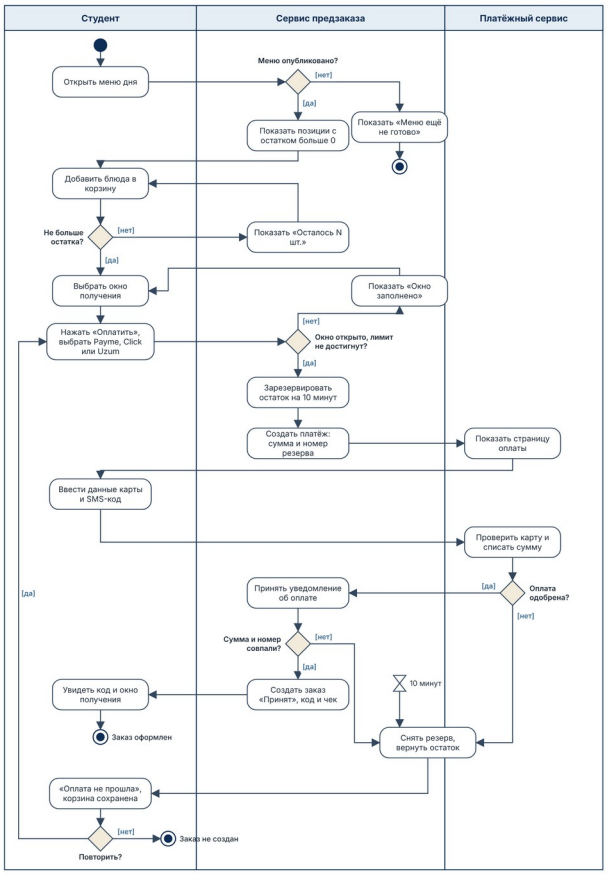


Рисунок 3.4 — Диаграмма деятельности «Оформить и оплатить заказ» (UML)

На диаграмме три дорожки: «Студент», «Сервис предзаказа» и «Платёжный сервис». Диаграмма имеет 5 альтернативных ветвей. Первая ветвь: меню ещё не опубликовано, и студент видит сообщение «Меню ещё не готово». Вторая ветвь: остаток меньше нужного количества, и сервис показывает «Осталось N шт.». Третья ветвь: окно заполнено, и студент выбирает другое окно. Четвёртая ветвь: платёжный сервис отклоняет оплату или сумма не совпала. Пятая ветвь: через 10 минут сервис снимает резерв и возвращает остаток. Во всех ветвях ошибки корзина сохраняется, и студент может повторить оплату.

Вывод. Диаграмма показывает все ошибки оформления заказа, которые команда нашла в интервью и в процессе «как будет». По этим ветвям команда напишет тест-кейсы ТС-013…ТС-017 на этапе 4.

3.2.3. Диаграмма последовательности: оплата заказа

Оплата — самая рискованная часть системы: в ней участвуют внешний сервис и деньги студента. Рисунок 3.5 показывает сообщения между шестью участниками в порядке времени. Номера сообщений помогают ссылаться на шаги.

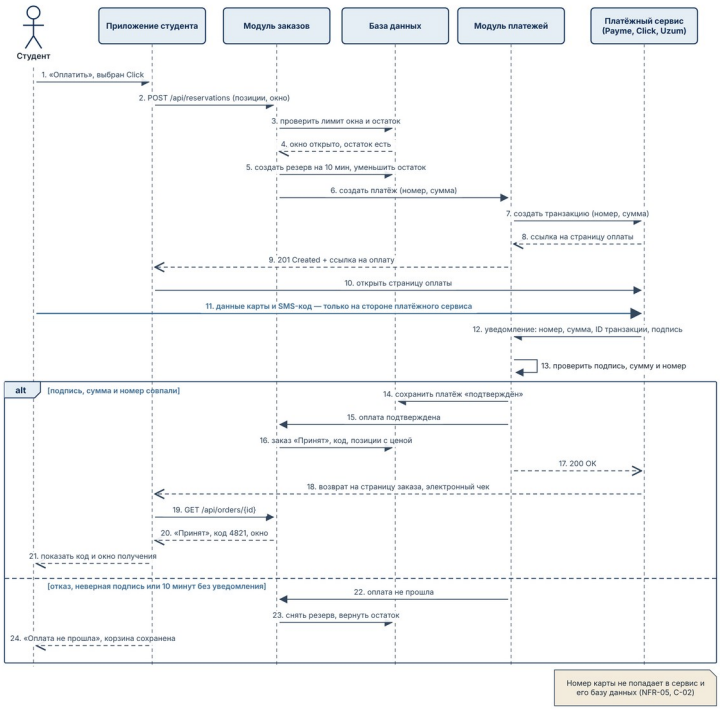


Рисунок 3.5 — Диаграмма последовательности «Оплатить заказ» (UML)

Приложение студента отправляет запрос POST /api/reservations (сообщение 2). Модуль заказов в одной транзакции проверяет лимит окна и остаток, резервирует остаток на 10 минут и создаёт платёж (сообщения 3–6). Студент переходит на страницу платёжного сервиса и вводит данные карты только там (сообщение 11). Платёжный сервис отправляет уведомление с подписью, а модуль платежей проверяет подпись, сумму и номер резерва (сообщение 12). Фрагмент alt имеет две ветви: если проверка успешна, модуль заказов создаёт заказ «Принят» и код получения; если пришёл отказ, подпись неверна или прошло 10 минут, сервис снимает резерв и показывает «Оплата не прошла».

Вывод. Номер карты не попадает в сервис и в его базу данных (NFR-05, C-02). Заказ появляется только после проверенного уведомления (FR-08), поэтому сбой не создаёт неоплаченный заказ.

3.2.4. Диаграмма классов предметной области

Диаграмма классов показывает понятия предметной области, их атрибуты и связи. Рисунок 3.6 показывает 12 классов; названия классов совпадают с терминами глоссария. По этой диаграмме команда построила схему базы данных в разделе 3.3.

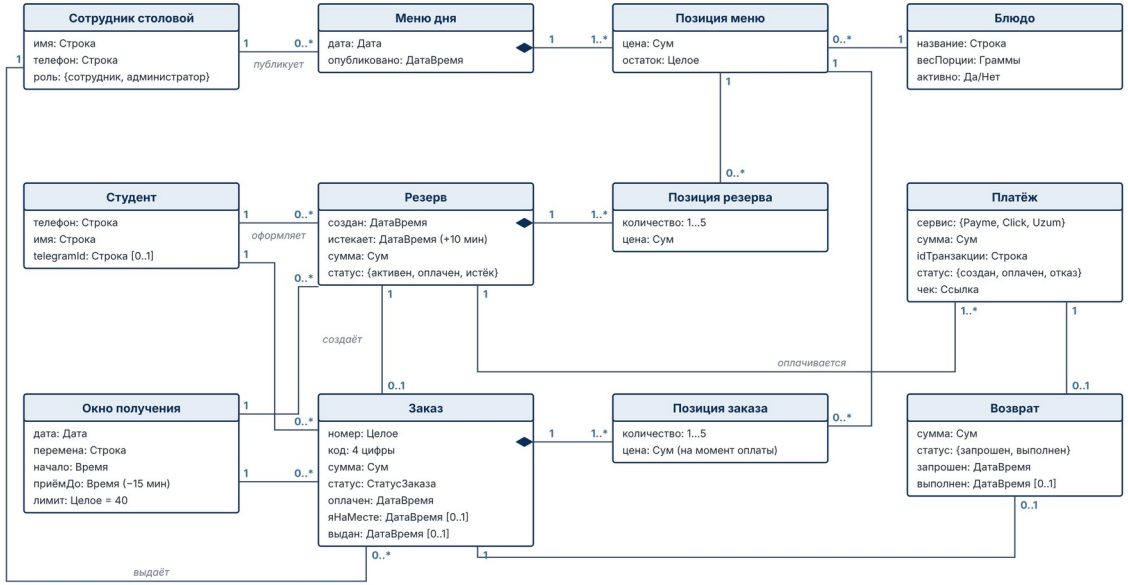


Рисунок 3.6 — Диаграмма классов предметной области (UML)

Композиция (закрашенный ромб) показывает, что часть не существует без целого: «Меню дня» — «Позиция меню», «Резерв» — «Позиция резерва», «Заказ» — «Позиция заказа». Кратность 1 — 0..1 между классами «Резерв» и «Заказ» показывает, что заказ появляется только после оплаты резерва, а истёкший резерв остаётся без заказа. Класс «Окно получения» имеет атрибуты «лимит = 40» и «приёмДо» (начало окна минус 15 минут). Класс «Позиция заказа» хранит цену на момент оплаты. Класс «Сотрудник столовой» имеет атрибут «роль» со значениями «сотрудник» и «администратор», поэтому две роли One-Pager описывает один класс.

Вывод. Класс «Резерв» решает противоречие требований FR-08 и FR-09: остаток держится до оплаты, а заказ появляется только после оплаты.

3.2.5. Согласованность моделей

Четыре вида диаграмм согласованы между собой и с бэклогом. Три роли называются одинаково в историях, на диаграмме вариантов использования (рисунок 3.3), в дорожках диаграммы деятельности (рисунок 3.4), среди участников диаграммы последовательности (рисунок 3.5) и в классах (рисунок 3.6): Студент, Сотрудник столовой, Администратор столовой. Понятия «резерв», «заказ», «окно получения» и «код

получения» есть в глоссарии, на диаграммах и в таблицах базы данных под одним названием. Например, резерв на 10 минут (FR-09) — это часть варианта UC-05, действие «Зарезервировать остаток на 10 минут», сообщение 5 диаграммы последовательности, класс «Резерв» и таблица reservations.

3.3. Модель данных

Рисунок 3.7 показывает физическую схему базы данных: первичные ключи (PK), внешние ключи (FK), типы полей и связи. Схема создана из SQL-скрипта schema.sql, который команда запустила в PostgreSQL 16.

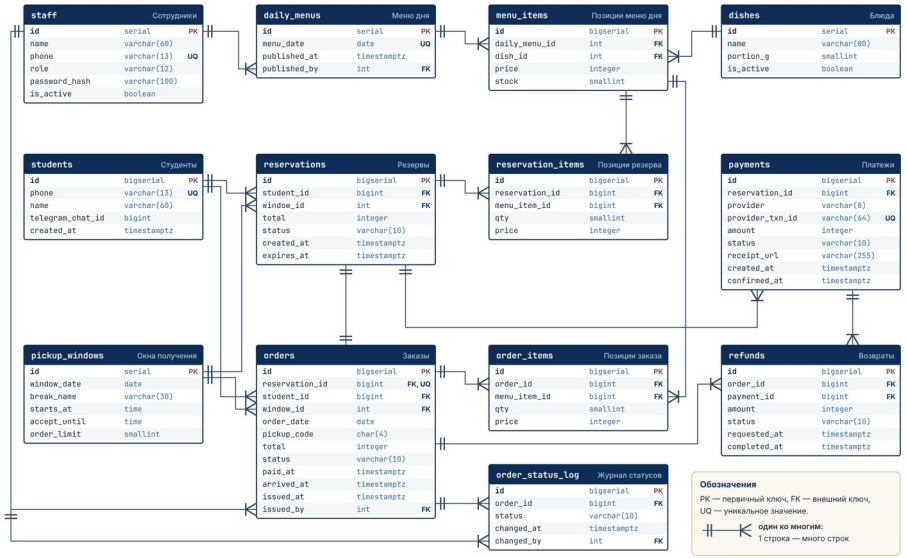


Рисунок 3.7 — Физическая схема базы данных (PostgreSQL 16)

В схеме 13 таблиц и 18 внешних ключей. Все связи, кроме одной, имеют тип 1:N: например, один заказ имеет много позиций заказа, одно окно — много заказов. Связь reservations — orders имеет тип 1:1, потому что из одного резерва появляется не больше одного заказа (UNIQUE на orders.reservation_id). Суммы хранятся в сумах как integer, потому что у сума нет дробной части в ценах столовой. Время хранится как timestamptz, поэтому часовой пояс не создаёт ошибок. SQL-скрипт создаёт 13 таблиц и 18 внешних ключей без ошибок.

Вывод. Схема выполняет требования задания к данным: больше 8 таблиц, таблица позиций заказа, первичные и внешние ключи, типы полей и связи 1:N.

3.3.1. Разбор схемы данных

Цена хранится в позиции заказа, потому что заказ — это запись о сделке, а не ссылка на меню. Администратор меняет цены в меню (US-16), например поднимает цену плова с 24 000 до 26 000 сум. Если позиция заказа хранит только ссылку на menu_items, то вчерашний заказ после изменения покажет новую цену, сумма заказа перестанет совпадать с выпиской платёжного сервиса, а отчёт продаж (FR-22) покажет неверную выручку. Поэтому поле order_items.price получает цену в момент оплаты и после этого не меняется. Это не нарушение нормализации по смыслу: цена в момент оплаты — отдельный факт, который зависит от заказа, а не от позиции меню.

Остальные отступления от третьей нормальной формы нужны для двух требований. Поля orders.total и reservation_items.price нужны для сверки денег: задача BG-3 и требование NFR-04 требуют 0 расхождений с выпиской, а сверка по готовой сумме не зависит от пересчёта позиций и даёт одно число для сравнения с выпиской. Поля menu_items.stock и orders.status нужны для скорости чтения: меню и панель столовой читают остаток и статус в каждом запросе, а пересчёт по журналу при 100 пользователях нарушает порог NFR-01. Цена денормализации — риск расхождения копии и источника. Команда закрывает этот риск транзакциями и тестом этапа 4, который сравнивает orders.total с суммой qty × price по каждому заказу.

3.4. Архитектура

3.4.1. Обоснование стиля и границ модулей

Модульный монолит — самый простой стиль, который выполняет требования к оплате и к стоимости. Нагрузка продукта небольшая: около 250 обедов в большую перемену (A-04, модельные данные) и 50 онлайн-заказов по цели BG-2. Один сервер VPS с 2 vCPU и 4 ГБ памяти обслуживает такую нагрузку, а ограничение C-01 требует, чтобы этот сервер стоял в Узбекистане. Главный аргумент — оплата. Резерв остатка, платёж и заказ должны меняться в одной транзакции базы данных, иначе сбой между шагами создаёт заказ без оплаты или оплату без заказа (NFR-04). В монолите это одна транзакция PostgreSQL, а в микросервисах — распределённая транзакция с компенсациями.

Границы модулей команда провела по владению данными: каждый модуль владеет своими таблицами и читает таблицы другого модуля только через его функции. Модуль «Заказы и резерв» владеет таблицами резерва и заказа, модуль «Платежи и возвраты» — таблицами payments и refunds, модуль «Меню и остатки» — таблицами меню. Граница между заказами и платежами проходит по месту риска: модуль платежей общается с внешним сервисом, и его можно вынести в отдельный сервис без переписывания остальных модулей. Модуль «Выдача» отделён от модуля заказов, потому что им пользуется другая роль на другом устройстве, и его экран меняется чаще.

3.4.2. Уровни системы

Рисунок 3.8 показывает уровни системы, модули сервиса и внешние системы справа. Номера требований на рисунке показывают, какой модуль выполняет какое требование.

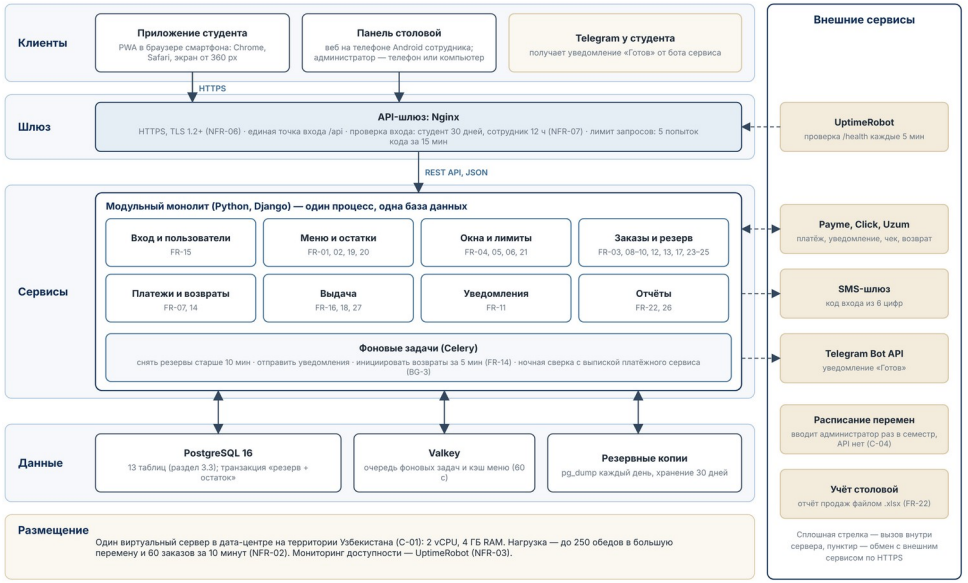


Рисунок 3.8 — Архитектура: уровни, модули и внешние системы

Уровень клиентов содержит приложение студента (PWA в браузере смартфона), панель столовой на телефоне сотрудника и Telegram у студента. Шлюз Nginx принимает только HTTPS с TLS 1.2 и выше, проверяет вход и ограничивает число попыток ввода кода. Уровень сервисов — модульный монолит на Django с 8 модулями и фоновыми задачами Celery. Уровень данных содержит PostgreSQL 16 для заказов, Valkey для кэша меню и очереди задач и ночные резервные копии. Справа стоят шесть внешних систем: платёжные сервисы Payme, Click и Uzum, SMS-шлюз, Telegram Bot API, мониторинг UptimeRobot, расписание перемен и учёт столовой.

Вывод. Все компоненты работают на одном сервере в Узбекистане (C-01).

3.4.3. Как работает система: путь заказа

Сервис состоит из шести процессов на одном сервере. Таблица 3.2 показывает, что делает каждый процесс и когда он работает.

Таблица 3.2 — Процессы на сервере

Процесс	Что делает	Когда работает
Nginx	Принимает HTTPS, отдаёт файлы приложения PWA, передаёт запросы API в Django	Всегда
Gunicorn + Django (4 рабочих процесса)	Выполняет REST API: восемь модулей сервиса (рисунок 3.8)	Всегда
PostgreSQL 16	Хранит 13 таблиц; каждая оплата идёт в одной транзакции	Всегда
Valkey	Кэш меню дня и очередь фоновых задач	Всегда
Celery worker	Выполняет фоновые задачи: сообщение в Telegram, возврат, SMS, отчёт .xlsx	Когда в очереди есть задача
Celery beat	Запускает задачи по расписанию: удаляет истёкшие резервы (каждую минуту), сверку с выпиской (каждую ночь)	По расписанию

Запрос студента или сотрудника идёт по цепочке Nginx → Django → PostgreSQL или Valkey. Медленные действия (сообщение в Telegram, возврат, SMS) Django не выполняет сам, а ставит в очередь Valkey; задачу выполняет Celery worker. Поэтому ответ на кнопку приходит быстро, даже если внешний сервис отвечает медленно. Ночная резервная копия pg_dump запускается по cron и хранится 30 дней.

Вывод. Шесть процессов разделяют быструю работу (ответ на запрос) и медленную работу (внешние сервисы и задачи по расписанию).

Таблица 3.3 показывает путь одного заказа через систему: от меню до выдачи и сверки. Для каждого шага таблица называет модуль, данные и требование, которое шаг выполняет.

Таблица 3.3 — Путь одного заказа через систему

Шаг	Что происходит	Модуль и данные	Требования
1. Меню	Студент открывает приложение. Django берёт меню дня из кэша Valkey; если кэша нет, читает PostgreSQL и кладёт меню в кэш	Меню и остатки: daily_menus, menu_items	FR-01, FR-02, NFR-01
2. Окно	Студент выбирает окно. Сервис показывает только окна, где приём открыт и лимит не набран	Окна и лимиты: pickup_windows	FR-04…FR-06
3. Резерв	Студент нажимает «К оплате». В одной транзакции сервис проверяет остаток, уменьшает его и создаёт резерв на 10 минут	Заказы и резерв: reservations, reservation_items	FR-03, FR-09
4. Оплата	Сервис создаёт платёж и открывает страницу Payme, Click или Uzum. Студент вводит карту там, а не в нашем приложении	Платежи: payments	FR-07, NFR-05
5. Уведомление об оплате	Платёжный сервис присылает уведомление. Сервис проверяет подпись и сумму. В одной транзакции резерв становится заказом с ценами и кодом из 4 цифр. Повторное уведомление не создаёт второй заказ: provider_txn_id уникален	Платежи → Заказы: payments, orders, order_items	FR-08, FR-12, FR-23, NFR-02, NFR-04
6. Истёкший резерв	Если оплаты нет 10 минут, Celery beat удаляет резерв и возвращает остаток в меню	Заказы и резерв, Меню и остатки	FR-09
7. Панель столовой	Сотрудник видит заказ в своём окне и меняет статус. Каждое изменение записывается в журнал статусов	Выдача, Заказы: order_status_log	FR-16, FR-17, NFR-07

Шаг	Что происходит	Модуль и данные	Требования
8. «Готов»	При статусе «Готов» Django ставит задачу в очередь; Celery worker отправляет сообщение в Telegram не позднее 30 секунд	Уведомления: Telegram Bot API	FR-10, FR-11
9. Выдача	Студент нажимает «Я на месте», сотрудник вводит код. Сервис записывает время и ставит статус «Выдан»	Выдача: orders	FR-18, FR-27, NFR-09
10. Отмена	До статуса «Готовится» студент отменяет заказ. Celery worker начинает возврат через платёжный сервис не позднее 5 минут	Платежи и возвраты: refunds	FR-13, FR-14
11. Конец дня	Ночью Celery beat сверяет оплаты с выпиской платёжного сервиса, cron делает резервную копию, администратор берёт отчёт	Отчёты, Платежи	FR-22, FR-26, NFR-04

Две транзакции защищают деньги студента. Первая (шаг 3) не даёт продать одну порцию двум студентам. Вторая (шаг 5) создаёт заказ только вместе с подтверждённой оплатой, поэтому оплаченный заказ не теряется, а неоплаченный не появляется. Медленные внешние сервисы работают в фоне (шаги 6, 8, 10, 11), поэтому студент и сотрудник не ждут их ответа.

Вывод. Путь заказа проходит через шесть модулей из восьми и выполняет 20 требований. Риски внешних систем закрывают проверка подписи, уникальный ID транзакции и ежедневная сверка с выпиской.

3.5. Прототип интерфейса

Прототип показывает главный путь студента и экран сотрудника столовой. В прототипе 7 связанных экранов, среди них экран ошибки оплаты. Прототип сделан в Figma, ссылка открывается без входа (режим «Anyone with the link can view») и дана на титульном листе и в приложении Ж: ⟦ссылка на Figma — добавит Салим⟧. Копия прототипа в HTML лежит в репозитории в папке /prototype.

3.5.1. Итоги проверки прототипа

Пять участников споткнулись в трёх местах: на незнакомом слове, на неясном состоянии денег и на мелком элементе. Незнакомое слово «Окно» остановило двух участников, и один из них (У5) поэтому не уложился в 90 секунд. Неясное состояние денег после отказа оплаты встретили три участника из пяти: экран сообщал об ошибке, но не говорил, что деньги не списаны. Это та же боль, которую линия эмоций показала на этапе «Заказ и оплата» (раздел 2.4). Мелкий код и кнопка внизу без ясной подписи замедлили задачу Т3, а отсутствие клавиши стирания создало ошибку на панели сотрудника.

Проверка показала, что главный путь работает без помощи: все пять участников выполнили задачи Т1–Т3, а на панели сотрудника прошли 10 выдач из 10 (модельные данные, приложение Д). Поэтому версия 2 не меняет набор экранов и переходы, а меняет только слова, состояние денег и размер элементов. Больше всех от исправлений получает студент, который платит впервые: экран ошибки теперь отвечает на его главный вопрос — списаны ли деньги. Сотруднику столовой важна клавиша стирания: без неё неверная цифра кода останавливает выдачу у стойки, а порог NFR-09 даёт на выдачу только 10 секунд.

3.6. Выбор платформы и стоимость владения

3.6.1. Варианты реализации

Команда сравнила три варианта по шести факторам темы 7: уникальность процесса, срок до пилота, бюджет, интеграции, команда после запуска и рост. Вариант А — no-code на Bubble: пилот через 1–1,5 месяца, но резерв, код и отметка «Я на месте» делаются только обходными путями, а серверы платформы за рубежом. Вариант Б — готовая SaaS-платформа заказов: пилот через 2–3 недели, но в ней нет резерва (FR-09), кода (FR-12) и отметки (FR-27), а хранение данных в Узбекистане не гарантировано. Вариант В — своя разработка на открытом стеке: пилот через 3–4 месяца, зато выполняются все требования Must, интеграция с Payme, Click и Uzum (C-02) и хранение данных в Узбекистане (C-01). Решающими стали ограничение C-01 и требования Must, поэтому дальше команда сравнивает стоимость, чтобы знать цену этого выбора. Лицензии всех компонентов варианта В открытые (MIT, BSD, PostgreSQL License); вместо Redis взят Valkey с лицензией BSD.

3.6.2. Стоимость владения за три года

Стоимость владения (TCO) — все расходы на продукт за выбранный период. Таблица 3.4 считает TCO за 3 года по пяти группам. Тема 7 даёт четыре группы: внедрение, регулярные платежи, поддержка и доработки, скрытые расходы. Команда разделила группу «внедрение» на две: «разработка» (код и тесты) и «внедрение в столовой» (подключение оплаты, установка, обучение, пилот и приёмка). Так видно, сколько стоит продукт и сколько стоит его запуск в столовой. Час работы — 100 000 сум, курс — 1 USD = 12 500 сум.

Таблица 3.4 — Стоимость владения за 3 года (TCO), сум

Статья расходов	А. No-code	Б. SaaS	В. Своя разработка
1. Разработка			
Код и настройка (100 000 сум/ч)	12 000 000 (120 ч: конструктор и платёжный API)	2 000 000 (20 ч: настройка платформы)	40 000 000 (400 ч кода)
Тесты по тест-кейсам ТС	входит в 120 ч	входит в 20 ч	8 000 000 (80 ч)
2. Внедрение в столовой			
Подключение Payme, Click, Uzum: заявки, тестовые платежи, боевые ключи	1 600 000 (16 ч)	0 (делает поставщик)	1 600 000 (16 ч)
Установка: сервер, домен, HTTPS, резервные копии	200 000 (2 ч: домен в Bubble)	200 000 (2 ч: домен у поставщика)	800 000 (8 ч)
Обучение сотрудников и администратора	200 000 (2 ч)	200 000 (2 ч)	200 000 (2 ч)
Сопровождение пилотной недели	1 000 000 (10 ч)	1 000 000 (10 ч)	1 000 000 (10 ч)
Приёмка столовой по тест-кейсам, акт	400 000 (4 ч)	400 000 (4 ч)	400 000 (4 ч)
Подключение платёжного сервиса через поставщика	—	800 000 (8 ч)	—
3. Регулярные платежи			
Платформа или сервер	31 050 000 ($69 в месяц)	21 600 000 (600 000 в месяц)	9 000 000 (VPS 250 000 в месяц)
Домен	600 000	входит в подписку	600 000
SMS-коды (около 500 SMS в месяц)	1 800 000	входит в подписку	1 800 000

Статья расходов	А. No-code	Б. SaaS	В. Своя разработка
Сертификат и мониторинг	0	0	0 (Let’s Encrypt, UptimeRobot)
4. Поддержка и доработки			
Поддержка (6 ч в месяц; SaaS — 2 ч в месяц)	21 600 000	7 200 000	21 600 000
Развитие (40 ч в год; no-code — 30 ч в год)	9 000 000	0 (менять нельзя)	12 000 000
5. Скрытые расходы			
Резерв 10 % от групп 1–4: простои, обновления, непредвиденные работы	7 945 000	3 340 000	9 700 000
Итого TCO за 3 года	**87 395 000**	**36 740 000**	**106 700 000**

Самая крупная группа варианта В — разработка: 48 млн сум, 45 % TCO. Внедрение в столовой стоит 4 млн сум у варианта В, 3,4 млн сум у варианта А и 2,6 млн сум у варианта Б: почти все эти часы нужны в любом варианте, потому что сотрудников надо обучить, пилот — сопроводить, а столовая должна принять работу. У вариантов А и Б главные статьи другие: у Bubble — подписка (31 млн сум за 3 года), у SaaS — подписка и поддержка. Поддержка 6 часов в месяц стоит 21,6 млн сум за 3 года и одинакова для вариантов А и В: в обоих случаях команда сама исправляет ошибки и обновляет компоненты. Регулярные платежи варианта В самые низкие (11,4 млн сум), потому что открытые компоненты не требуют лицензионных платежей. Каждая статья разобрана в разделе 3.6.3.

Вывод. Вариант В дороже варианта А на 19,3 млн сум и дороже варианта Б на 70 млн сум за 3 года. Главная часть разницы — разработка: 48 млн сум у варианта В против 12 млн сум у варианта А; внедрение в столовой стоит 2,6–4 млн сум у всех вариантов.

3.6.3. Сравнение вариантов и разбор стоимости владения

Варианты стоят на разных точках спектра темы 7. Вариант А (no-code) собирает продукт в визуальном конструкторе Bubble без программирования. Вариант Б (SaaS) — готовая платформа заказов по подписке, размещённая у поставщика. Вариант В — своя разработка на открытом стеке: React, Django, PostgreSQL и Valkey. Чем правее вариант по спектру, тем больше гибкости и тем выше цена запуска. Для нашего продукта гибкость обязательна: резерв остатка на 10 минут, код получения и отметка «Я на месте» — требования Must, и без них продукт не выполняет задачи BG-1 и BG-4.

Разработка — главная группа варианта В. Команда оценила 400 часов кода по объёму бэклога (73 story points первой версии) и 80 часов тестов по тест-кейсам TC-001…TC-070 и TC-101…TC-110. Для варианта А 120 часов — настройка конструктора и подключение платёжного API через доработку, для варианта Б 20 часов — настройка готовой платформы. Миграция данных не нужна ни в одном варианте, потому что сейчас у столовой нет учёта.

Внедрение в столовой — работа после разработки и до первого рабочего дня. Таблица 3.5 показывает шаги внедрения варианта В по порядку.

Таблица 3.5 — Шаги внедрения варианта В в первой столовой

Шаг	Что делает команда	Кто участвует	Часы	Результат
1. Подключение оплаты	Подаёт заявки в Payme, Click и Uzum от имени столовой, проводит тестовые платежи, вводит боевые ключи	Команда, бухгалтер столовой	16	Тестовый платёж каждого сервиса прошёл и вернулся

Шаг	Что делает команда	Кто участвует	Часы	Результат
2. Установка	Готовит VPS, домен, HTTPS, ночную резервную копию и мониторинг	Команда	8	Сайт открывается по HTTPS, копия восстановлена
3. Обучение	Показывает панель сотруднику и администратору, раздаёт памятку на 1 странице	Сотрудник, администратор	2	Сотрудник выдал 3 тестовых заказа сам
4. Пилотная неделя	Дежурит в столовой в обеденные перемены, исправляет ошибки	Команда, сотрудник	10	Список ошибок закрыт
5. Приёмка	Проходит со столовой тест-кейсы Must, подписывает акт	Администратор столовой	4	Акт приёмки

Внедрение занимает 40 часов, или 4 млн сум, и идёт после разработки. Обучение одной столовой длится 2 часа, как в юнит-экономике раздела 1.3; обучение следующих столовых входит в стоимость их привлечения. Для вариантов А и Б установка сводится к настройке домена (2 часа). В варианте Б подключение API делает поставщик, но команда готовит для него заявку на платёжный сервис (8 часов).

Вывод. Разработка создаёт продукт, а внедрение запускает его в конкретной столовой. Поэтому вложения проекта — сумма двух групп: 48 + 4 = 52 млн сум.

Итог сравнения: вариант В дороже по TCO, но только он выполняет ограничение С-01, интеграцию с Payme, Click и Uzum (С-02) и все требования Must. Вариант Б самый дешёвый, но покупает не наш продукт. Вариант А нарушает закон о персональных данных. Поэтому команда выбирает вариант В, а риск высокой стоимости внедрения закрывает пилотом: пилот в одной столовой проверяет спрос до подключения следующих столовых.

3.6.4. Окупаемость

Три столовые при целевом объёме дают 150 заказов в день и 3 960 000 сум маржи за учебный месяц. Постоянные расходы трёх столовых — 2 133 000 сум, потому что поддержка растёт на 2 часа в месяц на каждую новую столовую. Результат — 1 827 000 сум за учебный месяц, или 49,3 млн сум за 27 учебных месяцев трёх лет. Значит, три столовые возвращают 52 млн сум вложений за 29 учебных месяцев, то есть чуть больше чем за 3 года; с резервом скрытых расходов 9,7 млн сум срок растёт ещё примерно на полгода. При спросе на 30 % ниже цели три столовые возвращают вложения примерно за 9 лет, а для возврата за 3 года нужно 5 столовых. Этот вывод совпадает с выводом юнит-экономики раздела 1.3.4.

Выводы по главе 3

Процесс «как будет» убирает пять проблем процесса «как есть» шестью изменениями, и каждое изменение работает на одну из задач BG-1…BG-4. Больше всех выигрывает студент, а сотрудник столовой получает новую работу до перемены, и поэтому обучение сотрудника входит в договор о пилоте. Сотрудник больше не считает остаток «на глаз» и не принимает наличные у стойки: остаток ведёт система, а оплата приходит до перемены.

Четыре вида UML-диаграмм согласованы между собой, с бэклогом и со схемой базы данных: 21 вариант использования покрывает 19 историй, а роли, классы и таблицы называются одинаково. Схема данных содержит 13 таблиц и 18 внешних ключей; шесть денормализованных полей имеют причину и правило, а цена в позиции заказа хранит факт сделки. Поэтому изменение цены в меню не меняет старые заказы и отчёт продаж.

Архитектура — модульный монолит на Django с 8 модулями, PostgreSQL 16 и Valkey на сервере в Узбекистане. Каждое требование NFR-01…NFR-10 имеет архитектурное решение. Путь заказа из 11 шагов показывает, как модули работают вместе: две транзакции защищают остаток и оплату, а медленные внешние сервисы работают в фоне через Celery. Прототип из 7 экранов выполнил пороги NFR-08 и NFR-09 на модельной проверке, а шесть найденных проблем исправлены в версии 2. Главный риск архитектуры — зависимость от платёжных сервисов; его закрывают проверка подписи уведомления, уникальный ID транзакции и ежедневная сверка с выпиской.

Своя разработка на открытом стеке стоит 106,7 млн сум за 3 года: 48 млн сум разработки, 4 млн сум внедрения, остальное — платежи, поддержка и резерв. Она дороже вариантов А и Б, но только она выполняет ограничения С-01, С-02 и требования Must. Вложения 52 млн сум возвращаются чуть больше чем за 3 года при трёх столовых и целевом спросе, что совпадает с юнит-экономикой главы 1. Вариант Б дешевле в три раза, но не выполняет требования Must, а вариант А нарушает ограничение С-01, поэтому их стоимость не даёт альтернативы.

Заключение

Работа связывает идею, требования и проект одной цепочкой трассировки. Проблема очереди подтверждена реальными данными малой выборки и модельными данными этапа 2. Бизнес-модель с комиссией 5 % работает при трёх столовых и больше: одна столовая даёт убыток 280 000 сум за учебный месяц, а три столовые возвращают вложения в разработку и внедрение (52 млн сум) чуть больше чем за 3 года. Спецификация версии 2.1 содержит 27 функциональных и 10 нефункциональных требований без оценочных слов, бэклог — 19 историй первой версии на 73 story points. Глава 3 даёт процесс «как будет», согласованные UML-модели, схему из 13 таблиц, модульный монолит с описанием пути заказа по шагам и прототип из 7 экранов. Выбрана своя разработка на открытом стеке: TCO 106,7 млн сум за 3 года, из них разработка 48 млн сум и внедрение 4 млн сум.

Вторая задача — тестирование. Команда напишет тест-кейсы TC-001…TC-070 по критериям приёмки и по ветвям диаграмм деятельности, а тест-кейсы TC-101…TC-110 — по порогам нефункциональных требований. Третья задача — валидация прототипа: команда нарисует экраны входа, администратора и отчёта, добавит повтор заказа и раздел «Помощь» и повторит проверку на пяти реальных студентах по сценарию приложения Д. Четвёртая задача — исправить замечания защиты промежуточного контроля и записать их в приложение И.

Список использованных источников

1) ISO/IEC/IEEE 29148:2018. Systems and software engineering — Life cycle processes — Requirements engineering.
2) ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.
3) Object Management Group. Business Process Model and Notation (BPMN), Version 2.0.2. — 2014.
4) Object Management Group. Unified Modeling Language (UML), Version 2.5.1. — 2017.
5) Закон Республики Узбекистан «О персональных данных» от 02.07.2019 № ЗРУ-547.
6) Остервальдер А., Пинье И. Построение бизнес-моделей: настольная книга стратега и новатора. — М.: Альпина Паблишер, 2011.
7) Osterwalder A., Pigneur Y., Bernarda G., Smith A. Value Proposition Design. — Hoboken: Wiley, 2014.
8) Вигерс К., Битти Д. Разработка требований к программному обеспечению. 3-е изд. — М.: Русская редакция; СПб.: БХВ-Петербург, 2014.
9) Cohn M. User Stories Applied: For Agile Software Development. — Boston: Addison-Wesley, 2004.
10) Nielsen J. 10 Usability Heuristics for User Interface Design. — Nielsen Norman Group, 1994 (обновлено 2020). — URL: https://www.nngroup.com/articles/ten-usability-heuristics/
11) PostgreSQL 16 Documentation. — The PostgreSQL Global Development Group. — URL: https://www.postgresql.org/docs/16/
12) Django Documentation. — Django Software Foundation. — URL: https://docs.djangoproject.com/
13) Bubble. Pricing. — URL: https://bubble.io/pricing (дата обращения: 02.11.2026).
14) Valkey. — The Linux Foundation. — URL: https://valkey.io/

15) Ботиров С. Р. Программная инженерия: материалы курса, темы 1–7; задания этапов 1–3 и задание на промежуточный контроль. — Ташкент: Университет Пучон в городе Ташкенте, 2026.

Приложение А. Протоколы интервью (модельные данные)

Таблица А.1 даёт краткие протоколы интервью П1–П5 для раздела 2.3. Интервью — модельные данные: команда составила ответы по своему опыту и по данным этапа 1. Гайды и правила интервью даны в полной версии работы.

Таблица А.1 — Протоколы интервью П1–П5 (модельные данные)

№	Респондент	Дата, место, интервьюер	Цитаты	Факты	Выводы
П1	Студент 1 курса; обедает в столовой 2–3 раза в неделю	20.10.2026, холл 2 этажа, Руслан (С)	«Я на перемене просто не успеваю: пока протиснешься к стойке, уже звонок». «Чаще всего беру самсу и доедаю по дороге на пару».	В прошлый понедельник пришёл в малую перемену в 11:20. Ждал около 6 минут, ел на ходу. За 2 недели 2 раза ушёл без обеда. Платит переводом на карту.	H1 и H2 подтверждены. Окна получения нужны и на малых переменах (US-03).
П2	Студентка 4 курса; обедает 4–5 раз в неделю	21.10.2026, столовая, Салим (В)	«Плов к большой перемене обычно уже заканчивается, остаётся только суп». «Я специально прихожу в первую перемену, чтобы что-то досталось».	За 2 недели 3 раза не было нужного блюда. Меню узнаёт только у стойки. В кафе платит через Payme и Click.	H3 подтверждена. Нужны меню с остатком до 08:00 (US-01, US-16) и резерв (FR-09).
П3	Студент 3 курса; часто пропускает обед	21.10.2026, онлайн-звонок, Вика (А)	«Если у стойки человек десять, я даже не встаю в очередь — иду в Daika, хоть там и дороже». «Заплатить заранее нормально, но планы меняются — не хочу терять деньги».	На прошлой неделе пропустил обед в столовой 3 раза из 4. Дважды обедал в кафе рядом, около 100 000 сум за раз.	H2 подтверждена. Добавлена отмена заказа с возвратом (US-08, Should).
П4	Сотрудник раздачи; стаж больше 5 лет	22.10.2026, столовая до открытия, Вика (А)	«В перемену я одна: и накладываю, и деньги проверяю, и все кричат одновременно». «Если буду знать заказы за 15 минут, успею разложить».	В большую перемену выдаёт 200–250 обедов. Переводы проверяет по SMS на своём телефоне Android. Wi-Fi нет, мобильный интернет есть. Остаток блюд считает «на глаз».	H6 изменена: панель — на телефоне сотрудника. Порог 15 минут для FR-05. A-04: около 250 обедов.
П5	Руководитель (арендатор) столовой	23.10.2026, кабинет столовой, Руслан (С)	«Я не знаю, сколько у меня продаётся плова. Считаю по тому, что осталось вечером». «Месяц попробуем бесплатно. Если пойдёт — процент, но не фиксированную плату».	Учёта продаж и кассы нет. Готов выделить полку для онлайн-заказов. Кухня собирает до 40 заказов на одно окно.	H5 подтверждена с условием: пилотный месяц бесплатно, затем комиссия 5 % (A-07). Лимит окна — 40 (US-17).

Приложение Б. Опрос (модельные данные)

Таблица Б.1 даёт результаты опроса B1–B10 на 36 ответах. Результаты — модельные данные; реальный опрос 09.10.2026 описан в разделе 1.2.

Таблица Б.1 — Результаты опроса (модельные данные)

Показатель	Значение
Ответов всего / прошли фильтр	41 / 36
B2: обедают 2 раза в неделю и чаще	81 % (29 из 36)
B3: в последний раз пришли в малую перемену / в большую	31 % / 53 %
B4: ждали 5 минут и больше	56 % (20 из 36): 5–7 мин — 39 %, больше 7 мин — 17 %
B5: уходили без обеда хотя бы 1 раз за 2 недели	64 % (23 из 36)
B6: нужного блюда не было хотя бы 1 раз	75 % (27 из 36)
B7: в последний раз платили наличными / переводом на карту	42 % / 53 %
B8: платили онлайн за месяц	86 % (31 из 36); Payme 61 %, Click 47 %, Uzum 31 %
B9: интернет в здании стабильный / с перебоями / нет	67 % / 28 % / 6 %
B10: курс 1 / 2 / 3 / 4 / другое	8 / 9 / 12 / 5 / 2

Приложение В. Функциональные требования

Таблица В.1 даёт все 27 функциональных требований спецификации версии 2.1 с источником и историей бэклога. Раздел 2.5 описывает их группы, а матрица трассируемости (таблица 2.2) связывает их с вариантами использования, экранами и тест-кейсами.

Таблица В.1 — Функциональные требования

ID	Требование	История
FR-01	Система должна показывать меню текущего дня: название, цену, вес порции и остаток позиции.	US-01
FR-02	Система должна скрывать из меню позицию с остатком 0.	US-01
FR-03	Система должна позволять студенту добавить позицию в корзину в количестве от 1 до 5.	US-02
FR-04	Система должна показывать окна получения текущего дня по расписанию перемен.	US-03
FR-05	Система должна закрывать приём заказов на окно за 15 минут до начала окна.	US-03
FR-06	Система должна делать окно недоступным, когда число заказов в окне равно лимиту.	US-03
FR-07	Система должна принимать оплату заказа через Payme, Click или Uzum.	US-04
FR-08	Система должна создавать заказ только после подтверждения оплаты платёжным сервисом.	US-04
FR-09	Система должна резервировать остаток позиций заказа на время оплаты, не дольше 10 минут.	US-04
FR-10	Система должна показывать студенту текущий статус заказа.	US-05
FR-11	Система должна отправлять студенту уведомление не позднее 30 секунд после статуса «Готов».	US-06

ID	Требование	История
FR-12	Система должна присваивать каждому оплаченному заказу код получения из 4 цифр, уникальный в пределах дня.	US-07
FR-13	Система должна позволять студенту отменить заказ до статуса «Готовится».	US-08
FR-14	Система должна инициировать возврат полной суммы не позднее 5 минут после отмены.	US-08
FR-15	Система должна регистрировать студента по номеру телефона с подтверждением кодом из SMS.	US-11
FR-16	Система должна показывать сотруднику оплаченные заказы, сгруппированные по окнам получения.	US-12
FR-17	Система должна позволять сотруднику менять статус заказа в порядке «Принят» → «Готовится» → «Готов» → «Выдан».	US-13
FR-18	Система должна находить заказ текущего дня по коду получения и записывать время выдачи.	US-14
FR-19	Система должна позволять сотруднику поставить остаток позиции 0 одним действием.	US-15
FR-20	Система должна позволять администратору добавлять, изменять и снимать с продажи позиции меню.	US-16
FR-21	Система должна позволять администратору задать лимит заказов для каждого окна.	US-17
FR-22	Система должна формировать отчёт продаж за выбранный период: число заказов, сумму и число порций по позициям.	US-18
FR-23	Система должна сохранять каждый оплаченный заказ с временем оплаты, суммой, составом и статусом.	US-04
FR-24	Система должна показывать студенту его заказы за последние 30 дней.	US-10
FR-25	Система должна добавлять в корзину позиции прошлого заказа, у которых остаток больше 0.	US-09
FR-26	Система должна показывать администратору среднее и максимальное время получения по каждому окну за день.	US-19
FR-27	Система должна записывать время, когда студент отмечает «Я на месте».	US-07

Приложение Г. Описание ключевых таблиц базы данных

Приложение описывает две таблицы, через которые проходит оплаченный заказ: заказы (таблица Г.1) и позиции заказа (таблица Г.2). Схема всех 13 таблиц дана на рисунке 3.7, полный SQL-скрипт — в репозитории (models/schema.sql).

Таблица Г.1 — orders (Заказы)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	Номер заказа
reservation_id	bigint	FK, UNIQUE	Резерв (1:1)
student_id	bigint	FK → students.id	Студент
window_id	int	FK → pickup_windows.id	Окно получения
order_date	date	NOT NULL	День заказа
pickup_code	char(4)	NOT NULL	Код; UNIQUE (order_date, pickup_code)
total	integer	NOT NULL	Сумма, сум
status	varchar(10)	NOT NULL, CHECK	«Принят», «Готовится», «Готов», «Выдан», «Отменён»
paid_at	timestamptz	NOT NULL	Время оплаты

Поле	Тип	Ограничение	Описание
arrived_at	timestamptz	NULL	«Я на месте» (FR-27)
issued_at	timestamptz	NULL	Время выдачи (FR-18)
issued_by	int	FK → staff.id, NULL	Кто выдал

Таблица Г.2 — order_items (Позиции заказа)

Поле	Тип	Ограничение	Описание
id	bigserial	PK	
order_id	bigint	FK → orders.id	Заказ
menu_item_id	bigint	FK → menu_items.id	Позиция меню
qty	smallint	NOT NULL, CHECK 1…5	Количество
price	integer	NOT NULL	Цена на момент оплаты

Приложение Д. Сценарий проверки прототипа

Проверку проводят два человека: ведущий даёт задачи, наблюдатель записывает время и трудности. Каждый участник проходит задачи Т1–Т4 по одной (таблица Д.1) и думает вслух. Ведущий не подсказывает; если участник просит помощи, попытка считается неуспешной. Наблюдатель записывает время каждой задачи и трудности участника. Результаты этапа 3 — модельные данные (раздел 3.5).

Таблица Д.1 — Задачи проверки прототипа

Задача	Текст для участника	Успех	Мера
Т1	Закажите плов и зелёный чай на большую перемену. Оплатите через Click.	Участник нажал «Оплатить» на экране Э-4	NFR-08: ≤ 90 с у 4 из 5
Т2	Оплата не прошла. Оплатите заказ другим сервисом.	Участник открыл экран заказа Э-6	Успех 5 из 5
Т3	Найдите код получения и отметьте, что вы у точки выдачи.	Участник нажал кнопку отметки	Успех 5 из 5
Т4	Вы — сотрудник столовой. Выдайте заказ по коду, который назовёт ведущий (2 раза).	Участник нажал «Выдать заказ» для верного заказа	NFR-09: ≤ 10 с

Приложение Е. Организация работы команды

Таблица Е.1 даёт правила работы команды из устава (раздел 1.1).

Таблица Е.1 — Правила работы команды

Пункт	Договорённость
Капитан	Фаткулбаянов Руслан: представляет команду, сдаёт работы и публикует ссылки в чате курса.
Канал связи	Группа команды в Telegram. Ответ на сообщение — в течение 4 часов в дневное время.
Регулярная встреча	Среда, 15:00, 30 минут: онлайн (Telegram или Google Meet) или в университете.
Решения	Обсуждение в группе. Если согласия нет — голосование, решает большинство (2 из 3). По содержанию продукта последнее слово у Product Owner, по срокам — у менеджера проекта.

Пункт	Договорённость
Срыв срока	Участник предупреждает команду не позднее чем за 24 часа. Менеджер проекта перераспределяет задачу. Повторный срыв команда обсуждает на встрече.
Определение готовности	Документ готов, когда он соответствует заданию, проверен вторым участником, загружен в папку команды и его карточка на доске стоит в колонке «Готово».

Приложение Ж. Ссылки и исходные файлы

Таблица Ж.1 даёт ссылки на репозиторий и исходные файлы работы.

Таблица Ж.1 — Ссылки и исходные файлы

Ресурс	Ссылка или путь	Что содержит
Репозиторий команды	https://github.com/Zombik333/team1-predzakaz-obeda	Документы этапов, модели, прототип, тест-кейсы, устав команды
Прототип в Figma	⟦ссылка на Figma — добавит Салим⟧	7 экранов версии 2, переходы (раздел 3.5)
/docs	Папка репозитория	Проектная работа PDF и DOCX, документы этапов 1–3
/models	Папка репозитория	BPMN: bpmn_asis, bpmn_tobe1, bpmn_tobe2; UML: uml_usecase, uml_activity_order, uml_activity_pickup, uml_sequence_payment, uml_class; db_schema; architecture; schema.sql; db_schema.dbml
/prototype	Папка репозитория	prototype.html — копия прототипа в HTML; снимки экранов Э-1…Э-7
/tests	Папка репозитория	Список тест-кейсов ТС-001…ТС-070 и ТС-101…ТС-110
/team	Папка репозитория	Устав команды и роли

Приложение И. История изменений

Таблица И.1 записывает главные изменения работы: что было, что стало, в каком разделе и по чьему замечанию. Строки с жёлтыми полями команда заполнит после уточнения замечаний защит. Полная история из 27 строк дана в полной версии работы.

Таблица И.1 — История изменений

Дата	Раздел	Что было	Что стало	По чьему замечанию
⟦дата защиты этапа 1⟧	⟦раздел⟧	⟦что было⟧	⟦что исправлено⟧	⟦Преподаватель, защита этапа 1 — уточнить у команды⟧
02.11.2026	1.4, 2.1.2	Четыре бизнес-цели BG-1…BG-4 одного уровня	Одна цель BG-0 и четыре задачи BG-1…BG-4; номера задач сохранены	Преподаватель, защита этапа 2
02.11.2026	Презентация этапа 2	ФТ и НФТ на общем слайде; пустые зоны; ярко-рыжий цвет	Отдельные слайды для 27 ФТ и 10 НФТ; пустоты заполнены; спокойная палитра (тёмно-синий, песочный)	Преподаватель, защита этапа 2
⟦неделя 7⟧	⟦раздел⟧	⟦что было⟧	⟦что исправлено⟧	⟦Преподаватель, защита этапа 3 — добавить после защиты⟧

Дата	Раздел	Что было	Что стало	По чьему замечанию
02.11.2026	Вся работа	Три отдельных документа этапов, своя нумерация и приложения в каждом	Одна работа: сквозная нумерация, общее содержание, приложения А–И	Задание на промежуточный контроль
02.11.2026	Вся работа	Таблицы и схемы без текста или с выводом в одну строку	У каждой таблицы и рисунка подводка, разбор и вывод; обязательные текстовые разделы глав	Задание на промежуточный контроль
02.11.2026	2.2, 3.1–3.2, 3.3, 3.5	Роль «Сотрудник раздачи» в BPMN, UML, RACI, задаче Т4; staff.role = «раздача»	Роль «Сотрудник столовой» во всех главах; staff.role = «сотрудник»	Задание на ПК: единые названия ролей
02.11.2026	1.2	Бизнес-модель: комиссия 5 % или фиксированная плата 500 000 сум, выбор после разговора	Комиссия 5 % после бесплатного пилотного месяца; фиксированная плата отклонена (П5)	Результаты этапа 2
02.11.2026	1.3	—	Новый раздел: холст бизнес-модели, профиль клиента и список «может быть», юнит-экономика, расчёт при спросе −30 %, точка безубыточности, три фильтра	Задание на ПК (тема 3)
02.11.2026	2.3, 2.4, 2.5.3, 3.5, прил. А, Б	Интервью, опрос, замеры и проверка прототипа без пометки	Пометка «модельные данные» в тексте и в названиях таблиц	Решение команды (R) на защите этапа 2
02.11.2026	2.4, прил. А	Персоны сотрудников с именем и возрастом; возраст в протоколах	Обобщённые персоны ролей без имени и возраста; протоколы без возраста	Решение команды: сотрудники не опрошены
02.11.2026	2.5.2	НФТ: порог и условия только в столбцах, без связи с BG и ТС	НФТ версии 2.1: одно предложение с порогом и условиями, столбцы пяти шагов, BG и ТС-101…ТС-110	Задание на ПК (метод пяти шагов, тема 5)
02.11.2026	2.7, 3.7	Две таблицы трассировки: 2.7 (BG, FR, US, ТС) и 3.7 (UC, экран, БД)	Одна матрица 2.7: US, приоритет, BG, FR, UC, экран, таблицы БД, ТС	Задание на ПК
02.11.2026	3.6	ТСО по статьям без групп; окупаемость «3 × 35,75 ≈ 107 млн > 102,3 млн»	ТСО по четырём группам темы 7; окупаемость тем же методом, что в 1.3: 3 столовые — около 3 лет, при спросе −30 % — 5 столовых	Задание на ПК: нет противоречий между главами
02.11.2026	Рисунки 2.1, 2.2 полной версии	Рыжий цвет точек и линий	Спокойная палитра: тёмно-синий и приглушённые цвета	Преподаватель: без ярких цветов
03.11.2026	3.6.2, 3.6.4	Группа «внедрение» — только разработка 48 млн сум; обучение — в резерве; ТСО В 102,3 млн сум	Отдельно «разработка» (48 млн сум) и «внедрение в столовой» (4 млн сум, 40 ч по шагам); ТСО В 106,7 млн сум; вложения 52 млн сум	Преподаватель, промежуточный контроль
03.11.2026	3.4.4	—	Как работает система: процессы на сервере и путь одного заказа по 11 шагам	Преподаватель, промежуточный контроль
03.11.2026	Вся работа	Полная версия: 101 страница, 88 таблиц и рисунков	Сокращённая версия: в главах остались объекты, которые требует задание; остальные таблицы и рисунки — в полной версии в репозитории	Преподаватель: объём около 40 страниц на весь семестр